

Trabajo de Fin de Grado

Diseño y desarrollo de un sistema empotrado basado en bus CAN con acceso inalámbrico para la monitorización de un vehículo automóvil

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Gorka Eymard Santana Cabrera

TUTORIZADO POR:

Antonio Carlos Domínguez Brito, Jorge Cabrera Gámez

Fecha [03/2026]

Agradecimientos

La realización de este Trabajo de Fin de Título no habría sido posible sin el apoyo y la colaboración de diversas personas e instituciones a las que deseo expresar mi más sincero agradecimiento.

En primer lugar, quiero agradecer a los profesores Antonio C. Domínguez Brito y Jorge Cabrera Gámez, por su dedicación, orientación y apoyo durante el desarrollo de este trabajo. Su experiencia y los consejos recibidos han sido fundamentales para encauzar el desarrollo de este proyecto en la dirección correcta.

Asimismo, agradezco al profesor Jorge Cabrera Gámez, tutor colaborador, su implicación y el valioso apoyo técnico aportado durante las distintas fases del trabajo. Sus indicaciones han contribuido de manera significativa a la calidad del resultado final.

De igual modo, deseo expresar mi gratitud a la División de Robótica y Oceanografía Computacional del Instituto Universitario SIANI de la Universidad de Las Palmas de Gran Canaria, por poner a disposición de este proyecto los medios materiales y el equipamiento necesarios para llevar a cabo el desarrollo experimental y las pruebas del sistema.

Por último, agradezco a todas las personas que, de una forma u otra, han brindado su apoyo durante este período, haciendo posible la culminación de este trabajo.

Resumen

La estructura de este documento es orientativa. Las **partes obligatorias (motivación y objetivos, competencias, aportaciones, alineamiento ODS, desarrollo, conclusiones y bibliografía)** deben estar recogidas, aunque pueden reordenarse o redistribuirse en base a las características del trabajo.

EL presente Trabajo de Fin de Título(TFT) desarrolla un sistema empotrado basado en bus CAN que trabaja en la capa de aplicación de OBDII permitiendo la realización de diagnóstico de vehículos. El sistema ha sido diseñado y validado tomando como referencia la arquitectura de comunicaciones de los vehículos del grupo VAG (Volkswagen Audi Group), cuya topología de *Gateway* central es representativa de los vehículos modernos de la plataforma PQ25.

Abstract

Ahora en inglés: This Final Degree Project (TFT) develops an embedded system based on the CAN bus that operates at the OBD-II application layer, enabling vehicle diagnostics. The system has been designed and validated against the communication architecture of Volkswagen Audi Group (VAG) vehicles, whose central *Gateway* topology is representative of modern PQ25 platform vehicles.

Índice general

Listings	V
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Descripción del sistema desarrollado	2
1.3. Estructura del documento	2
2. Estado actual y objetivos	4
2.1. Motivación y antecedentes	4
2.2. Objetivos	5
2.2.1. Objetivo general	5
2.2.2. Objetivos específicos	5
2.2.3. Alcance y limitaciones	6
3. Aportaciones del trabajo	7
3.1. Principales aportaciones	7
3.2. Competencias específicas desarrolladas	8
3.3. Relación con los Objetivos de Desarrollo Sostenible	9
4. Metodología de desarrollo	12
4.1. Metodología de desarrollo	12
4.2. Fases y tareas	13
4.2.1. Fase 1 — Análisis de requisitos (50 horas estimadas)	13
4.2.2. Fase 2 — Diseño rápido y construcción (75 horas estimadas)	13
4.2.3. Fase 3 — Evaluación, refinamiento e implementación (125 horas estimadas)	14
4.2.4. Fase 4 — Documentación y presentación (50 horas estimadas)	14
4.3. Desviaciones respecto a la planificación inicial	14
5. Desarrollo del sistema	16
5.1. Arquitectura general del sistema	16
5.2. Proceso de desarrollo iterativo	17
5.2.1. Fase 1: Validación del enlace CAN	17
5.2.2. Fase 2: Primer prototipo — simulador ECU y CLI de diagnóstico	18
5.2.3. Fase 3: Simulación avanzada, monitorización y persistencia	19

5.2.4.	Fase 4: Integración BLE y validación en vehículo real	20
5.3.	Protocolos de comunicación	22
5.3.1.	CAN — Controller Area Network (ISO 11898)	22
5.3.2.	ISO-TP — Transport Protocol (ISO 15765-2)	23
5.3.3.	OBD-II — On-Board Diagnostics (SAE J1979)	24
5.3.4.	BLE GATT y Nordic UART Service	25
5.4.	Simulador de ECU — Arduino MKR	26
5.4.1.	Hardware y configuración del bus CAN	26
5.4.2.	Modelo físico del motor	27
5.4.3.	Implementación de los modos OBD-II	27
5.4.4.	Mecanismos de simulación de ruido CAN	29
5.4.5.	Inyección de DTCs por interrupción hardware	29
5.5.	Herramienta de diagnóstico — Raspberry Pi	30
5.5.1.	Arquitectura software	30
5.5.2.	Capa de transporte CAN e ISO-TP	31
5.5.3.	Módulo de logging en SQLite	31
5.5.4.	Monitor de datos en tiempo real	32
5.5.5.	Servidor BLE GATT	32
5.6.	Aplicación móvil — React Native	33
5.6.1.	Arquitectura de la aplicación	33
5.6.2.	Pantallas y funcionalidades	33
5.6.3.	Comunicación BLE	34
5.7.	Integración y pruebas	35
5.7.1.	Escenarios de prueba	35
5.7.2.	Resultados de las pruebas de integración	36
6.	Resultados	37
6.1.	Resultados obtenidos	37
6.2.	Validación sobre vehículo real	37
6.3.	Validación funcional del protocolo OBD-II	38
6.4.	Métricas de rendimiento de la comunicación	38
6.5.	Comportamiento ante condiciones adversas	39
6.6.	Grado de consecución de los objetivos	40
7.	Conclusiones y trabajo futuro	41
7.1.	Conclusiones	41
7.2.	Líneas de trabajo futuro	42
7.3.	Reflexión sobre el uso de herramientas de inteligencia artificial	43
Anexo A	Código fuente del simulador Arduino	44
Anexo B	Código fuente de la herramienta Python	48
Anexo C	Código fuente de la aplicación móvil	53
Anexo D	Esquema de la base de datos SQLite	57

Listings

1.	Constantes de CAN, ISO-TP y ruido — <code>ecu_protocol.h</code>	44
2.	Estructura <code>VehicleData</code> y funciones de codificación OBD-II	45
3.	Interrupción hardware para inyección de DTCs e inicialización	45
4.	Inyección de DTC por interrupción hardware con debounce	46
5.	Mecanismos de ruido CAN dentro del procesador de mensajes	46
6.	Respuesta VIN con secuencia ISO-TP multi-frame	47
7.	Interfaz <code>ITransport</code> — contrato de la capa de transporte	48
8.	<code>IsoTpTransport</code> — implementacion concreta sobre <code>python-can</code> + <code>can-isotp</code> .	49
9.	<code>SqliteDataLogger</code> — esquema, buffer de escritura y modo WAL	50
10.	<code>LoggingTransport</code> — patron decorador sobre <code>ITransport</code>	51
11.	UUIDs del servicio NUS y configuracion de la conexion BLE	53
12.	Flujo de conexion: GATT, MTU, servicios, suscripcion y autenticacion . . .	53
13.	Procesamiento del buffer NDJSON recibido por notificaciones BLE	54
14.	Sistema de peticion-respuesta con timeout y chunking por MTU	55
15.	Ping periodico y desconexion automatica por inactividad	56
16.	Esquema completo de la base de datos <code>diagnostics.db</code>	57

Índice de algoritmos

Índice de figuras

5.1.	Arquitectura general del sistema de diagnóstico vehicular.	16
5.2.	Arduino IDE — monitor serie durante la fase de validación CAN. Se aprecia la confirmación de inicialización del bus ([OK] CAN Bus at 500 kbps) y el log de cada trama enviada.	17
5.3.	Sesión SSH en la Raspberry Pi mostrando la salida de <code>candump can0</code> y una petición manual con <code>cansend</code> durante la validación del enlace CAN.	18
5.4.	Monitor serie del Arduino durante la Fase 2: cada petición recibida ([RX]) y su respuesta ([TX]) se registran indicando modo, PID y bytes de datos. . . .	18
5.5.	CLI de diagnóstico (<code>cli.py</code>) en la Fase 2: menú interactivo con opciones de live data y lectura de DTCs, ejecutado en la Raspberry Pi vía SSH.	19
5.6.	Monitor serie del Arduino en la Fase 3: se aprecian los logs de ruido ([DROP], [NRC]), los eventos de ignición ([IGN] Key ON) y la inyección de DTCs por interrupción hardware ([DTC] Injected P0300).	20
5.7.	CLI en modo de monitorización continua (opción 6): actualización periódica de los cinco PIDs principales con timestamp, valores decodificados y unidades, ejecutado vía SSH en la Raspberry Pi.	20
5.8.	Pantallas <i>Connection</i> (izquierda) y <i>Dashboard</i> (derecha) de la aplicación móvil durante las pruebas de integración BLE.	22
5.9.	Pantallas <i>DTCs</i> (izquierda) y <i>Logs</i> (derecha) de la aplicación móvil. La pantalla de DTCs muestra un código inyectado por interrupción hardware; la pantalla de Logs muestra el historial de sesiones almacenado en SQLite. . . .	22
5.10.	Flujo de transmisión ISO-TP multi-frame para la respuesta VIN (17 bytes). .	24
5.11.	Ciclo completo de una consulta OBD-II desde la aplicación móvil hasta la ECU.	25
5.12.	Estructura de capas del paquete BLE NUS.	25
5.13.	Flujo de establecimiento de sesión BLE GATT entre la aplicación móvil y la Raspberry Pi.	26
6.1.	Sistema montado en el vehículo real: cable OBD-II conectado al puerto de diagnóstico, Raspberry Pi con el servidor activo y teléfono móvil con la aplicación mostrando datos de telemetría en tiempo real.	38

Índice de cuadros

3.1. Grado de relación del TFT con los Objetivos de Desarrollo Sostenible de la Agenda 2030.	10
5.1. Estructura de una trama CAN 2.0A (<i>Standard Frame</i>). El campo ID de 11 bits determina la prioridad: a menor valor numérico, mayor prioridad en el bus.	22
5.2. Tipos de trama ISO-TP (ISO 15765-2). El byte PCI (en negrita) codifica en sus cuatro bits superiores el tipo de trama y en los inferiores metadatos de longitud o control de flujo. El relleno 0xAA se usa para completar los 8 bytes del payload CAN.	23
5.3. Formato de petición y respuesta OBD-II (SAE J1979) encapsulados como payload ISO-TP sobre CAN. Se muestra el ejemplo de lectura de RPM (PID 0x0C).	24
6.1. PIDs del modo 0x01 validados en el banco de ensayo.	38
6.2. Latencia extremo a extremo de una consulta OBD-II (100 muestras, sin ruido CAN).	39
6.3. Evaluación del grado de consecución de los objetivos específicos.	40
1. Fragmento de la tabla samples — sesión de ejemplo.	58

Capítulo 1

Introducción

1.1. Contexto y motivación

El sector del automóvil ha experimentado durante las últimas décadas una transformación profunda que ha llevado al vehículo moderno a convertirse en una plataforma de computación distribuida sobre ruedas. Si a mediados del siglo XX un automóvil podía describirse en términos puramente mecánicos, hoy un turismo de gama media incorpora entre 30 y 100 Unidades de Control Electrónico (ECU, del inglés *Electronic Control Unit*), cada una de ellas responsable de gestionar un subsistema específico del vehículo: desde la inyección de combustible y la gestión del motor hasta el control de tracción, los sistemas de ayuda a la conducción o el infoentretenimiento. Esta proliferación de sistemas electrónicos ha mejorado drásticamente la seguridad, la eficiencia energética y la experiencia del conductor, pero ha introducido al mismo tiempo una complejidad técnica que plantea nuevos retos en materia de diagnóstico y mantenimiento.

Para que todas estas ECUs puedan intercambiar información de forma fiable y en tiempo real, los fabricantes adoptaron a partir de los años noventa el bus CAN (*Controller Area Network*), un protocolo de comunicaciones serie desarrollado originalmente por Robert Bosch GmbH en 1983 y estandarizado bajo la norma ISO 11898. El bus CAN permite que múltiples nodos compartan un mismo par de cables trenzados, con un mecanismo de arbitraje no destructivo basado en la prioridad del identificador de mensaje y una robusta señalización diferencial que lo hace especialmente resistente al ruido electromagnético propio del entorno vehicular.

Sobre esta infraestructura de comunicaciones, la industria desarrolló el estándar OBD-II (*On-Board Diagnostics*, segunda generación), que desde el año 1996 es obligatorio en Estados Unidos y desde principios de los 2000 en Europa bajo la denominación EOBD. OBD-II define un conector físico normalizado de 16 pines accesible desde el interior del vehículo, así como un conjunto de servicios y parámetros estandarizados que permiten a cualquier herramienta de diagnóstico externa consultar el estado del motor, leer los códigos de avería almacenados en las ECUs o identificar el vehículo mediante su número de identificación (VIN). Sin embargo, el acceso a esta información en la práctica ha quedado históricamente restringido al ámbito

profesional: los escáneres de diagnóstico comerciales que implementan estos protocolos de forma completa tienen un coste elevado, requieren cableado dedicado y, en muchos casos, ofrecen interfaces poco intuitivas orientadas exclusivamente al técnico especializado.

Este escenario pone de manifiesto una oportunidad clara: desarrollar una solución alternativa, de bajo coste y acceso inalámbrico, basada en plataformas de prototipado electrónico ampliamente disponibles, que permita acceder a la información diagnóstica del vehículo de forma portable e intuitiva. Esta es precisamente la motivación central de este Trabajo de Fin de Título.

1.2. Descripción del sistema desarrollado

El presente trabajo aborda el diseño y desarrollo de un sistema empujado de diagnóstico vehicular compuesto por tres componentes principales que trabajan de forma integrada.

El primer componente es un **simulador de ECU** implementado sobre una plataforma Arduino MKR con shield CAN. Este simulador reproduce el comportamiento de una ECU real en el bus CAN, respondiendo a peticiones OBD-II estándar con datos de telemetría generados mediante un modelo físico simplificado del motor: RPM, velocidad, temperaturas, presiones y códigos de avería, entre otros. El simulador incluye además mecanismos de inyección de ruido CAN y generación de tráfico de fondo, diseñados para validar la robustez del sistema de diagnóstico frente a condiciones adversas del bus.

El segundo componente es la **herramienta de diagnóstico**, ejecutada sobre una Raspberry Pi conectada al bus CAN mediante un controlador MCP2515. Esta herramienta implementa la pila de protocolos completa: ISO-TP (ISO 15765-2) como capa de transporte sobre CAN, y OBD-II (SAE J1979) como capa de aplicación. La herramienta decodifica los parámetros recibidos, los persiste en una base de datos SQLite y los expone de forma inalámbrica a través de un servidor BLE GATT que implementa el servicio Nordic UART Service (NUS).

El tercer componente es una **aplicación móvil** desarrollada con React Native y Expo, compatible con Android e iOS, que se conecta a la Raspberry Pi vía Bluetooth Low Energy. La aplicación ofrece al usuario una interfaz para la monitorización en tiempo real de los parámetros del vehículo, la consulta y gestión de códigos de avería, el acceso al historial de sesiones almacenadas y una consola de comandos directa.

El conjunto del sistema constituye una plataforma de diagnóstico OBD-II portable, inalámbrica y de bajo coste, validada sobre un banco de pruebas basado en Arduino antes de su uso sobre hardware vehicular real.

1.3. Estructura del documento

El presente documento se organiza en siete capítulos cuyo contenido se resume a continuación.

El **Capítulo 2** recoge la motivación detallada del trabajo, el análisis de las soluciones existentes en el mercado y la definición de los objetivos que se pretenden alcanzar.

El **Capítulo 3** describe las principales aportaciones del trabajo, las competencias específicas del Grado de Ingeniería Informática desarrolladas durante su realización y la relación del proyecto con los Objetivos de Desarrollo Sostenible de la ONU.

El **Capítulo 4** presenta la metodología de desarrollo seguida, basada en prototipos incrementales, y detalla las fases y tareas en las que se estructuró el trabajo, incluyendo una valoración de las desviaciones respecto a la planificación inicial.

El **Capítulo 5** constituye el núcleo técnico de la memoria. Describe en detalle los protocolos de comunicación empleados (CAN, ISO-TP, OBD-II y BLE GATT), el diseño e implementación de cada uno de los tres componentes del sistema, y las pruebas de integración realizadas.

El **Capítulo 6** presenta los resultados obtenidos, incluyendo métricas de rendimiento y una evaluación del grado de consecución de los objetivos planteados.

El **Capítulo 7** recoge las conclusiones del trabajo, las líneas de trabajo futuro identificadas y una reflexión sobre el uso de herramientas de inteligencia artificial durante el desarrollo.

Finalmente, el documento incluye la bibliografía de referencia y cuatro anexos con fragmentos representativos del código fuente desarrollado y el esquema de la base de datos.

Capítulo 2

Estado actual y objetivos

2.1. Motivación y antecedentes

El acceso a los datos internos de un vehículo automóvil ha sido durante décadas un ámbito reservado casi exclusivamente a los talleres especializados y a los fabricantes de automóviles. Los dispositivos de diagnóstico profesionales, conocidos coloquialmente como escáneres o *scan tools*, permiten comunicarse con las ECUs del vehículo a través del puerto OBD-II y consultar en tiempo real una amplia variedad de parámetros operativos, leer y borrar códigos de avería, e incluso reprogramar ciertos módulos electrónicos. Sin embargo, el coste de estas herramientas, que en el segmento profesional puede oscilar entre varios cientos y varios miles de euros, junto con la complejidad de su operación, ha mantenido este tipo de diagnóstico fuera del alcance del usuario particular y del desarrollador aficionado.

En los últimos años han aparecido en el mercado soluciones de bajo coste que pretenden democratizar el acceso al diagnóstico OBD-II. El adaptador ELM327, desarrollado por ELM Electronics, es posiblemente el más extendido de estos dispositivos. Se trata de un circuito integrado que actúa como puente entre el bus CAN del vehículo y una interfaz serie, habitualmente accesible vía Bluetooth o Wi-Fi. Existen multitud de aplicaciones móviles compatibles con este adaptador, como Torque Pro o OBD Fusion, que permiten visualizar parámetros básicos del motor desde un teléfono inteligente. No obstante, estas soluciones presentan limitaciones relevantes: el protocolo propietario AT utilizado por el ELM327 introduce latencias significativas, la calidad de los clones económicos disponibles en el mercado es muy variable, y la mayoría de las implementaciones no ofrecen acceso completo al conjunto de modos y servicios OBD-II, ni permiten el registro estructurado de datos para su análisis posterior.

En el ámbito más cercano a este trabajo, el Instituto Universitario SIANI de la Universidad de Las Palmas de Gran Canaria, a través de su División de Robótica y Oceanografía Computacional, dispone de medios hardware basados en plataformas de prototipado electrónico de bajo coste, como la Raspberry Pi y controladores CAN basados en el circuito MCP2515. Esta infraestructura disponible constituyó el punto de partida natural para plantear una solución alternativa a los adaptadores comerciales existentes: implementar directamente la pila de

protocolos OBD-II sobre hardware de propósito general, con control total sobre cada capa del sistema y capacidad para extender sus funcionalidades en el futuro.

A lo anterior se añade el interés técnico que representa la arquitectura de comunicaciones de los vehículos modernos del grupo VAG (Volkswagen Audi Group), en particular los modelos basados en la plataforma PQ25, como el SEAT Ibiza de sexta generación. Estos vehículos implementan una topología de *Gateway* central que separa el bus CAN-Powertrain, donde se comunican las ECUs de motor y transmisión, del bus de diagnóstico accesible desde el conector OBD-II. Esta arquitectura impide el acceso directo al bus de motor mediante un adaptador CAN estándar y obliga a que toda comunicación de diagnóstico pase a través del módulo Gateway, lo que añade una capa de complejidad adicional que los dispositivos comerciales de bajo coste no siempre gestionan correctamente. El estudio de esta arquitectura y sus implicaciones prácticas constituye uno de los antecedentes directos de este trabajo.

2.2. Objetivos

A partir del contexto descrito, se definen los siguientes objetivos para este Trabajo de Fin de Título.

2.2.1. Objetivo general

Diseñar y desarrollar un sistema empotrado de diagnóstico vehicular basado en el protocolo OBD-II sobre bus CAN, con acceso inalámbrico vía Bluetooth Low Energy, de bajo coste económico y validado mediante un banco de pruebas basado en Arduino.

2.2.2. Objetivos específicos

1. **Implementar la pila de protocolos OBD-II completa:** desarrollar sobre Raspberry Pi una implementación funcional de ISO-TP (ISO 15765-2) como capa de transporte y OBD-II (SAE J1979) como capa de aplicación, cubriendo los modos de servicio 0x01 (datos en vivo), 0x03 (lectura de DTCs), 0x04 (borrado de DTCs) y 0x09 (información del vehículo, incluyendo VIN).
2. **Desarrollar un simulador de ECU:** implementar sobre Arduino MKR con shield CAN un simulador de ECU que responda a peticiones OBD-II estándar con datos de telemetría realistas, con el objetivo de disponer de un banco de pruebas reproducible y controlado que no dependa de la disponibilidad de un vehículo real.
3. **Diseñar mecanismos de simulación de condiciones adversas:** incorporar en el simulador Arduino la generación de ruido CAN (pérdida de tramas, latencia variable, respuestas negativas esporádicas) y tráfico de fondo aleatorio, para validar la robustez de la herramienta de diagnóstico.

4. **Proporcionar acceso inalámbrico mediante BLE:** implementar un servidor BLE GATT sobre la Raspberry Pi que exponga la funcionalidad de diagnóstico a través del servicio Nordic UART Service (NUS), eliminando la necesidad de conexión física entre el dispositivo de diagnóstico y el usuario.
5. **Desarrollar una aplicación móvil multiplataforma:** implementar una aplicación para Android e iOS con React Native que permita al usuario conectarse al sistema de diagnóstico vía BLE, monitorizar parámetros del vehículo en tiempo real, gestionar los códigos de avería y consultar el historial de sesiones almacenadas.
6. **Registrar y persistir los datos de diagnóstico:** implementar un sistema de logging estructurado en base de datos SQLite que almacene las sesiones, las muestras de telemetría y los comandos ejecutados, para su consulta y análisis posterior.

2.2.3. Alcance y limitaciones

El sistema desarrollado se centra en la implementación de los modos OBD-II estándar definidos por SAE J1979, sin abordar los servicios de diagnóstico propietarios de cada fabricante, que requieren herramientas específicas y, en muchos casos, acuerdos de licencia con el fabricante del vehículo. Asimismo, la validación experimental del sistema se realiza íntegramente sobre el banco de pruebas Arduino, dado que las restricciones de acceso al hardware vehicular real durante el período de desarrollo del trabajo no permitieron completar las pruebas sobre el vehículo objetivo. Las implicaciones de esta limitación y las líneas de trabajo futuro que abre se discuten en el Capítulo 7.

Capítulo 3

Aportaciones del trabajo

3.1. Principales aportaciones

Este trabajo presenta un conjunto de aportaciones técnicas que, consideradas en su conjunto, constituyen una plataforma de diagnóstico vehicular OBD-II completa, de código abierto y bajo coste, diferenciada de las soluciones comerciales existentes en varios aspectos relevantes.

Implementación directa de la pila de protocolos. A diferencia de los adaptadores comerciales de tipo ELM327, que abstraen el protocolo OBD-II mediante comandos AT propietarios, el sistema desarrollado implementa directamente las capas ISO-TP y OBD-II sobre hardware de propósito general. Esto proporciona control total sobre los tiempos de comunicación, la gestión de errores y la extensibilidad del sistema, sin depender de firmware cerrado de terceros.

Banco de pruebas reproducible basado en Arduino. El simulador de ECU desarrollado sobre Arduino MKR con shield CAN constituye una herramienta de validación independiente del vehículo real. El simulador implementa un modelo físico del motor, responde a los cuatro modos OBD-II estándar y permite reproducir condiciones adversas del bus CAN de forma controlada y repetible: pérdida de tramas, latencia variable, respuestas negativas esporádicas e inyección aleatoria de códigos de avería mediante interrupción hardware. Esta capacidad de simulación de ruido es una aportación que distingue este trabajo de implementaciones similares disponibles en la literatura.

Arquitectura software modular con separación de capas. La herramienta de diagnóstico implementada en Python sobre Raspberry Pi sigue un diseño basado en interfaces y capas desacopladas mediante inyección de dependencias. Esta arquitectura facilita la sustitución de cualquier componente (transporte, decodificador, logger) sin modificar el resto del sistema, y permite el uso de un transporte (*mock*) para el desarrollo y las pruebas sin necesidad de hardware CAN.

Acceso inalámbrico mediante BLE GATT con logging persistente. La integración de un servidor BLE GATT sobre Raspberry Pi, combinada con el registro estructurado de

datos en SQLite, permite no solo el acceso en tiempo real a los parámetros del vehículo, sino también la consulta posterior de sesiones históricas completas, incluyendo las tramas CAN en hexadecimal correspondientes a cada comando ejecutado.

Aplicación móvil multiplataforma. La aplicación React Native desarrollada opera sobre Android e iOS con una única base de código, implementa el protocolo JSON sobre NUS para la comunicación con la Raspberry Pi y ofrece una experiencia de usuario orientada tanto a la monitorización en tiempo real como al análisis de datos históricos.

3.2. Competencias específicas desarrolladas

La realización de este Trabajo de Fin de Título ha requerido la aplicación y el desarrollo de competencias específicas del Grado de Ingeniería Informática, plan 41, de la Universidad de Las Palmas de Gran Canaria. A continuación se describe la relación de las competencias más directamente implicadas con el trabajo realizado.

CI1 — Capacidad para diseñar, desarrollar, evaluar y asegurar la accesibilidad, ergonomía, usabilidad y seguridad de los sistemas, servicios y aplicaciones informáticas, así como de la información que gestionan. Esta competencia se ha desarrollado en el diseño de la aplicación móvil, prestando atención a la usabilidad de la interfaz de diagnóstico, y en la implementación de mecanismos de seguridad básicos en la comunicación BLE (autenticación por token, timeout de inactividad).

CI5 — Capacidad para especificar, diseñar, implementar y evaluar interfaces persona-computador que garanticen la accesibilidad y usabilidad de los sistemas, servicios y aplicaciones informáticas. Esta competencia se ha aplicado en el diseño e implementación de la aplicación móvil React Native, con sus cinco pantallas funcionales y el sistema de navegación por pestañas.

CI8 — Capacidad para diseñar y evaluar sistemas operativos y servidores, y aplicaciones y sistemas basados en computación distribuida. El sistema desarrollado es por naturaleza un sistema distribuido: Arduino, Raspberry Pi y dispositivo móvil son tres nodos que se comunican mediante protocolos bien definidos. La Raspberry Pi actúa simultáneamente como cliente CAN y como servidor BLE, gestionando concurrencia mediante hilos y comunicación asíncrona.

CI14 — Capacidad para diseñar sistemas de tiempo real y sistemas empotrados. Esta es la competencia más directamente relacionada con el trabajo. El simulador Arduino opera en tiempo real con interrupciones hardware, temporización precisa de las tramas -TP y gestión concurrente de múltiples tareas (broadcast CAN, respuesta a peticiones OBD-II, generación de ruido). La Raspberry Pi implementa monitorización continua con periodos de muestreo de 500 ms.

CI16 — Capacidad para diseñar y desarrollar sistemas de comunicaciones. El núcleo técnico del trabajo es precisamente la implementación de una pila de protocolos de

comunicaciones: CAN (ISO 11898), ISO-TP (ISO 15765-2), OBD-II (SAE J1979) y BLE GATT.

CI17 — Capacidad para diseñar y desarrollar sistemas de seguridad informática.

Esta competencia se ha aplicado de forma parcial en el diseño del mecanismo de autenticación BLE y en la gestión segura de las respuestas negativas UDS, que permiten al sistema rechazar peticiones no autorizadas o fuera de contexto.

CI18 — Capacidad para diseñar e implementar sistemas de tiempo real y sistemas empotrados.

En complemento con CI14, esta competencia abarca el diseño del flujo de control del simulador Arduino, la implementación del servidor BLE con asyncio en Python y la coordinación entre el hilo de monitorización continua y el hilo de atención a comandos BLE.

3.3. Relación con los Objetivos de Desarrollo Sostenible

Los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de Naciones Unidas establecen un marco global de referencia para orientar el desarrollo tecnológico hacia un impacto positivo en la sociedad y el medio ambiente. La tabla 3.1 recoge el grado de relación de este trabajo con cada uno de los 17 ODS.

A continuación se justifica el grado de relación asignado a los ODS con mayor vinculación con el trabajo.

ODS 9 — Industria, Innovación e Infraestructuras (Alto). El trabajo desarrolla tecnología e innovación aplicada al sector del mantenimiento automovilístico, un ámbito industrial con un impacto económico significativo. La implementación directa de protocolos de diagnóstico sobre hardware de bajo coste abre posibilidades de innovación en entornos donde las herramientas profesionales resultan inaccesibles económicamente.

ODS 12 — Producción y Consumo Responsables (Medio). El acceso asequible al diagnóstico vehicular fomenta el denominado *derecho a reparar*: un diagnóstico preciso y oportuno permite al propietario del vehículo tomar decisiones informadas sobre el mantenimiento, alargando la vida útil del vehículo y reduciendo la generación de residuos tecnológicos derivados de la sustitución prematura de componentes.

ODS 7 y 13 — Energía Asequible y Acción por el Clima (Medio). El estándar OBD-II incluye parámetros directamente relacionados con la eficiencia de la combustión y las emisiones de gases contaminantes, como el estado del sistema de combustible, los recortes de mezcla (*fuel trim*) o el estado del catalizador. El acceso a esta información facilita la detección temprana de averías que incrementan las emisiones, contribuyendo indirectamente a la reducción de la huella de carbono del transporte.

ODS 4 — Educación de Calidad (Medio). El carácter abierto y documentado del sistema desarrollado, junto con el banco de pruebas basado en Arduino, lo convierte en una

Cuadro 3.1: Grado de relación del TFT con los Objetivos de Desarrollo Sostenible de la Agenda 2030.

ODS	Grado de relación			
	0 No procede	1 Bajo	2 Medio	3 Alto
1 Fin de la Pobreza	X			
2 Hambre Cero	X			
3 Salud y Bienestar		X		
4 Educación de Calidad			X	
5 Igualdad de Género	X			
6 Agua Limpia y Saneamiento	X			
7 Energía Asequible y No Contaminante			X	
8 Trabajo Decente y Crecimiento Económico		X		
9 Industria, Innovación e Infraestructuras				X
10 Reducción de las Desigualdades	X			
11 Ciudades y Comunidades Sostenibles		X		
12 Producción y Consumo Responsables			X	
13 Acción por el Clima			X	
14 Vida Submarina	X			
15 Vida de Ecosistemas Terrestres	X			
16 Paz, Justicia e Instituciones Sólidas	X			
17 Alianzas para Lograr los Objetivos	X			

herramienta didáctica de alto valor para la enseñanza de protocolos de comunicación industrial, sistemas empotrados y desarrollo de software en capas, en el contexto de titulaciones de ingeniería informática y electrónica.

Capítulo 4

Metodología de desarrollo

4.1. Metodología de desarrollo

El desarrollo de este trabajo se ha llevado a cabo siguiendo una metodología basada en la construcción iterativa de prototipos, tal y como se estableció en la propuesta inicial del trabajo (TFT01). Esta elección metodológica se justifica por la naturaleza exploratoria del proyecto: al abordar la implementación directa de protocolos de comunicación vehicular sobre hardware de propósito general, sin disponer de precedentes directos en el grupo de trabajo, resultaba imprescindible disponer de un enfoque que permitiera validar hipótesis técnicas de forma temprana y ajustar el diseño en función de los resultados obtenidos en cada iteración.

La metodología basada en prototipos se estructura en cuatro fases principales: análisis de requisitos, diseño rápido y construcción del prototipo inicial, evaluación y refinamiento iterativo, y documentación y presentación. Estas fases no son estrictamente secuenciales sino que admiten solapamiento parcial: en particular, las fases de construcción y evaluación se ejecutaron de forma alternada durante la mayor parte del desarrollo, con ciclos cortos de implementación seguidos de pruebas sobre el banco de ensayo Arduino.

Este enfoque permitió, por ejemplo, detectar de forma temprana las limitaciones de la primera implementación del protocolo ISO-TP, que no contemplaba la gestión correcta del *Flow Control* en mensajes multi-frame, y corregirlas antes de proceder al desarrollo de las capas superiores. De igual modo, la decisión de incorporar el módulo de simulación de ruido CAN en el simulador Arduino surgió como respuesta a los resultados de las primeras pruebas de integración, en las que se constató la necesidad de validar la robustez del sistema frente a condiciones de bus no ideales.

4.2. Fases y tareas

4.2.1. Fase 1 — Análisis de requisitos (50 horas estimadas)

Esta fase inicial se dedicó al estudio de los fundamentos técnicos necesarios para abordar el desarrollo del sistema.

Tarea 1.1 — Estudio de la plataforma hardware. Se analizaron las características de la Raspberry Pi como plataforma de ejecución de la herramienta de diagnóstico, incluyendo el soporte de la interfaz SocketCAN en Linux, la compatibilidad con el controlador CAN MCP2515 y las opciones disponibles para el servidor BLE. Asimismo, se evaluó el Arduino MKR como plataforma del simulador, estudiando la librería CAN disponible y sus capacidades de temporización.

Tarea 1.2 — Estudio de los protocolos de comunicación CAN. Se realizó un análisis detallado de los estándares ISO 11898 (CAN), ISO 15765-2 (ISO-TP) y SAE J1979 (OBD-II), prestando especial atención a la estructura de las tramas, los mecanismos de control de flujo y el formato de codificación de cada parámetro. Se revisaron asimismo los conceptos fundamentales de UDS (ISO 14229-1) relevantes para la implementación de las respuestas negativas.

Tarea 1.3 — Estudio de Bluetooth Low Energy y librerías disponibles. Se estudiaron los conceptos fundamentales de BLE GATT, el servicio Nordic UART Service (NUS) y las opciones de implementación disponibles en Python para la Raspberry Pi, evaluando las librerías `bleak` y `bless`. En el lado de la aplicación móvil, se analizó la librería `react-native-ble-plx` y su compatibilidad con las características de escritura y notificación del servicio NUS.

Tarea 1.4 — Análisis de herramientas software. Se definió el conjunto de herramientas de desarrollo a utilizar: Python 3 con las librerías `python-can` y `can-isotp` para la capa de transporte CAN, SQLite para la persistencia de datos, React Native con Expo para la aplicación móvil, y el IDE de Arduino para el simulador. Se estableció asimismo la estructura del repositorio y el flujo de trabajo con control de versiones Git.

4.2.2. Fase 2 — Diseño rápido y construcción (75 horas estimadas)

Tarea 2.1 — Diseño de la arquitectura del sistema. Se definió la arquitectura general del sistema en tres capas: simulador Arduino, herramienta de diagnóstico en Python y aplicación móvil. Para la herramienta Python se diseñó una arquitectura modular basada en interfaces abstractas (`ITransport`, `IProtocolBuilder`, `IDataDecoder`, `IDiagnosticSession`), con inyección de dependencias para facilitar las pruebas. Se definieron asimismo el esquema de la base de datos SQLite y el protocolo de comandos JSON utilizado sobre BLE.

Tarea 2.2 — Implementación del primer prototipo funcional. Se desarrolló una primera versión operativa del sistema que incluía: el simulador Arduino respondiendo a peticiones OBD-II básicas en modo 0x01 (RPM, temperatura de refrigerante, velocidad), la

herramienta Python con transporte ISO-TP funcional y sesión de diagnóstico básica, y una primera versión de la aplicación móvil con pantalla de conexión BLE y visualización de datos en tiempo real.

4.2.3. Fase 3 — Evaluación, refinamiento e implementación (125 horas estimadas)

Esta fue la fase de mayor duración e intensidad técnica, en la que el prototipo inicial evolucionó de forma iterativa hasta el sistema final.

Tarea 3.1 — Extensión del simulador Arduino. Se amplió el simulador con soporte completo para los cuatro modos OBD-II (0x01, 0x03, 0x04, 0x09), implementando la respuesta VIN mediante la secuencia ISO-TP multi-frame (FF + CF). Se añadió el modelo físico de simulación del motor, con la lógica de aceleración, velocidad, temperaturas y consumo de combustible. Posteriormente se incorporaron los mecanismos de simulación de ruido CAN y la interrupción hardware para inyección aleatoria de DTCs.

Tarea 3.2 — Extensión de la herramienta Python. Se completó la herramienta con el módulo de logging en SQLite, el decorador `LoggingTransport`, el monitor de datos en tiempo real con hilo dedicado, la sesión de diagnóstico extendida (`LoggedDiagnosticSession`) y el servidor BLE GATT con el manejador de comandos JSON (`BtCommandHandler`).

Tarea 3.3 — Desarrollo completo de la aplicación móvil. Se implementaron las cinco pantallas de la aplicación (Connection, Dashboard, DTCs, Logs, Console), los almacenes de estado con Zustand, el adaptador BLE con reintentos y gestión de timeout, y el adaptador simulado (`MockAdapter`) para el desarrollo sin hardware.

Tarea 3.4 — Pruebas de integración. Se realizaron pruebas del sistema completo sobre el banco de ensayo, verificando el comportamiento ante los escenarios de ruido CAN configurados en el simulador, la correcta persistencia de los datos en SQLite y la estabilidad de la conexión BLE durante sesiones de monitorización prolongadas.

4.2.4. Fase 4 — Documentación y presentación (50 horas estimadas)

Tarea 4.1 — Redacción de la memoria. Redacción del presente documento, incluyendo la elaboración de las tablas y figuras de los protocolos de comunicación, la documentación del código fuente incluida en los anexos y la preparación del material de apoyo para la defensa.

4.3. Desviaciones respecto a la planificación inicial

La planificación inicial recogida en el TFT01 contemplaba una distribución de 300 horas totales, con especial peso en la fase de evaluación e implementación (125 horas). En líneas

generales, esta estimación resultó ajustada a la realidad del desarrollo, si bien se produjeron algunas desviaciones significativas que conviene señalar.

La implementación del protocolo ISO-TP requirió un esfuerzo mayor al previsto inicialmente, en particular en lo referente a la gestión del *Flow Control* y a la sincronización entre el envío de la secuencia FF+CF en el simulador Arduino y la recepción en la Raspberry Pi. Esta fase consumió aproximadamente un 20 % más de tiempo del estimado, que se compensó reduciendo el alcance inicial de la aplicación móvil, que en la propuesta contemplaba también visualización gráfica de tendencias temporales.

Por otra parte, la incorporación del módulo de simulación de ruido CAN no estaba prevista en la planificación original, pero se identificó como necesaria durante las primeras pruebas de integración. Su desarrollo se realizó dentro de la Fase 3, absorbiendo parte del tiempo reservado a las pruebas sobre el vehículo real, que finalmente no pudieron llevarse a cabo por las razones expuestas en la Sección 2.2.3.

La decisión de estructurar el código Python con una arquitectura de interfaces y capas desacopladas supuso una inversión inicial de tiempo superior a la contemplada en el diseño rápido, pero resultó rentable en las fases posteriores al facilitar considerablemente la extensión del sistema y la realización de pruebas con el transporte simulado.

Capítulo 5

Desarrollo del sistema

5.1. Arquitectura general del sistema

El sistema desarrollado se articula en torno a tres nodos de procesamiento independientes que se comunican mediante dos canales físicos distintos: el bus CAN entre el simulador Arduino y la herramienta de diagnóstico, y Bluetooth Low Energy entre la herramienta de diagnóstico y la aplicación móvil. La figura 5.1 ilustra la arquitectura global del sistema.

Ilustración 5.1: Arquitectura general del sistema de diagnóstico vehicular.

El **simulador de ECU** (Arduino MKR con shield CAN) reproduce el comportamiento de una unidad de control electrónico vehicular sobre el bus CAN a 500 kbps. Responde a peticiones OBD-II en los cuatro modos implementados, genera datos de telemetría mediante un modelo físico simplificado del motor, e introduce ruido controlado en el bus para validar la robustez del sistema.

La **herramienta de diagnóstico** (Raspberry Pi con controlador MCP2515) actúa como nodo central del sistema: implementa la pila de protocolos completa (ISO-TP + OBD-II), persiste los datos en SQLite y expone el servicio BLE GATT hacia la aplicación móvil. Internamente opera con dos hilos de ejecución concurrentes: el hilo de monitorización continua y el hilo de atención a comandos BLE.

La **aplicación móvil** (React Native) se conecta a la Raspberry Pi vía BLE e interactúa con el sistema a través de comandos JSON sobre el servicio Nordic UART Service. Ofrece una interfaz de usuario estructurada en cinco pantallas funcionales.

La separación de responsabilidades entre los tres componentes permite que cada uno sea sustituido o extendido de forma independiente. En particular, la herramienta Python puede operar con un transporte simulado (*mock*) sin necesidad de hardware CAN, y la aplicación móvil incorpora un adaptador simulado que permite su desarrollo sin conexión Bluetooth activa.

5.2. Proceso de desarrollo iterativo

El sistema se construyó de forma incremental en cuatro fases, cada una verificable de forma independiente antes de añadir la siguiente capa de funcionalidad. Este enfoque permitió detectar problemas en la capa de transporte antes de añadir la lógica de protocolo, y validar la lógica de protocolo antes de integrar la interfaz de usuario.

5.2.1. Fase 1: Validación del enlace CAN

El primer paso fue verificar que los dos nodos hardware —Arduino MKR y Raspberry Pi con CAN HAT RS485— podían intercambiar tramas CAN correctamente antes de introducir ninguna lógica de protocolo.

Arduino — transmisión básica CAN

El sketch inicial se apoyó en la librería `arduino-CAN` de Sandeep Mistry, que abstrae el controlador MCP2515 vía SPI. La inicialización del bus se reduce a una sola llamada, como se muestra en el Listado A del Anexo A:

- ✓ `CAN.begin(CAN_SPEED)` inicializa el controlador a 500 kbps según la constante definida en `ecu_protocol.h` (`#define CAN_SPEED 500E3`).
- ✓ Cada trama se envía con `CAN.beginPacket(id)`, `CAN.write(data, len)` y `CAN.endPacket()`.
- ✓ El Serial a 115200 baud reporta el resultado de cada operación, lo que permite supervisar el comportamiento desde el IDE de Arduino.

Ilustración 5.2: Arduino IDE — monitor serie durante la fase de validación CAN. Se aprecia la confirmación de inicialización del bus ([OK] CAN Bus at 500 kbps) y el log de cada trama enviada.

Raspberry Pi — recepción y herramientas SocketCAN

En la Raspberry Pi, la interfaz CAN se levanta mediante el subsistema SocketCAN de Linux:

```
sudo ip link set can0 up type can bitrate 500000
```

Las utilidades `candump` y `cansend` del paquete `can-utils` permitieron verificar la recepción de tramas y enviar peticiones manuales sin necesidad de ningún código Python:

```
candump can0
cansend can0 7DF#0201050000000000
```

La salida de `candump` confirma el identificador, la longitud y los bytes de datos de cada trama recibida, lo que permitió verificar que el filtro de aceptación del controlador MCP2515 estaba correctamente configurado.

Ilustración 5.3: Sesión SSH en la Raspberry Pi mostrando la salida de `candump can0` y una petición manual con `cansend` durante la validación del enlace CAN.

5.2.2. Fase 2: Primer prototipo — simulador ECU y CLI de diagnóstico

Con el enlace CAN validado se desarrolló el primer prototipo funcional: el Arduino comenzó a responder a peticiones OBD-II reales, y la Raspberry Pi pasó de `candump` a ejecutar una herramienta Python estructurada.

Arduino — respuesta a peticiones OBD-II

El simulador incorporó la función `processCANMessage()` que analiza el modo de servicio (primer byte de datos) y el PID solicitado (segundo byte) para construir la respuesta en el formato ISO-TP Single Frame. Los modos implementados en esta fase fueron `0x01` (datos en vivo, PIDs básicos) y `0x03` (lectura de DTCs preconfigurados).

La constante `ECU_CAN_ID = 0x7E0` define el identificador de petición escuchado, y `ECU_RESPONSE_ID = 0x7E8` el identificador de respuesta emitido, ambos definidos en `ecu_protocol.h` en concordancia con el estándar SAE J1979.

Ilustración 5.4: Monitor serie del Arduino durante la Fase 2: cada petición recibida (`[RX]`) y su respuesta (`[TX]`) se registran indicando modo, PID y bytes de datos.

Raspberry Pi — pila ISO-TP y CLI

En la Raspberry Pi se introdujeron las librerías `python-can` [7] y `can-isotp` [6], que gestionan respectivamente el acceso al bus SocketCAN y la segmentación/reensamblado ISO-TP. La clase `IsoTpTransport` encapsula ambas en la interfaz `ITransport` del sistema:

- ✓ `isotp.Address` configura los identificadores de tester (`0x7E0`) y ECU (`0x7E8`) en modo Normal 11 bits.
- ✓ `isotp.CanStack` gestiona de forma automática los Flow Control frames y el reensamblado de respuestas multi-frame.
- ✓ El timeout de recepción (`timeout=5.0s`) absorbe la latencia de la fase de ruido sin generar falsos errores.

Sobre este transporte se desarrolló `cli.py`, un menú interactivo que permitió ejecutar consultas OBD-II desde la línea de comandos sin necesidad de la aplicación móvil. El menú inicial ofrecía dos operaciones principales:

- ✓ **Opción 1 — Live data:** consulta instantánea de los cinco PIDs más relevantes (RPM, temperatura, velocidad, posición del acelerador, carga del motor).
- ✓ **Opción 3 — Read DTCs:** lectura de la lista de códigos de avería almacenados en la ECU simulada.

Ilustración 5.5: CLI de diagnóstico (`cli.py`) en la Fase 2: menú interactivo con opciones de live data y lectura de DTCs, ejecutado en la Raspberry Pi vía SSH.

5.2.3. Fase 3: Simulación avanzada, monitorización y persistencia

La tercera fase añadió tres mejoras simultáneas: el simulador Arduino incorporó mecanismos de ruido controlado e interrupciones hardware, y la herramienta Python extendió el CLI con monitorización continua y logging en SQLite.

Arduino — ruido CAN e interrupciones

Los parámetros de ruido se centralizaron en `ecu_protocol.h` como constantes compiladas:

- ✓ `CAN_NOISE_DROP_PCT = 2`: probabilidad de descarte de petición (simulación de colisión o degradación de señal).
- ✓ `CAN_NOISE_NRC_PCT = 1`: probabilidad de respuesta negativa UDS con NRC `0x22` (*conditionsNotCorrect*).
- ✓ `CAN_NOISE_LATENCY_MIN_MS = 1`, `CAN_NOISE_LATENCY_MAX_MS = 15`: retardo aleatorio antes de procesar cada petición.
- ✓ `CAN_NOISE_BG_TRAFFIC_PCT = 30`: probabilidad por ciclo de emitir una trama de tráfico de fondo con identificadores `0x280`, `0x480`, `0x320` o `0x520`.

Adicionalmente, se implementaron dos interrupciones hardware para simular eventos espontáneos del vehículo:

- ✓ **Pin D7 (ignición):** la ISR `onIgnitionButton()` activa el flag `ignitionPressed`; la función `handleIgnitionButton()` en el bucle principal conmuta el estado del motor y ajusta las RPM al ralentí (850 rpm) o a cero.
- ✓ **Pin D6 (fallo DTC):** la ISR `onDtcFaultButton()` activa `dtcFaultPressed`; `handleDtcFaultButton()` selecciona aleatoriamente uno de los 12 DTCs del pool, comprueba duplicados e inserta el código en `dtcList[]`.

El patrón ISR mínima + debounce en bucle principal garantiza que no se ejecuten operaciones bloqueantes (`Serial`, `delay`, acceso al bus CAN) dentro de la rutina de interrupción.

Ilustración 5.6: Monitor serie del Arduino en la Fase 3: se aprecian los logs de ruido ([DROP], [NRC]), los eventos de ignición ([IGN] Key ON) y la inyección de DTCs por interrupción hardware ([DTC] Injected P0300).

Raspberry Pi — monitorización continua y SQLite

La herramienta Python extendió el CLI con dos nuevas capacidades:

Monitorización continua (`live_data_monitor.py`): la clase `LiveDataMonitor` ejecuta un hilo *daemon* que itera sobre el conjunto de PIDs configurado, envía cada petición ISO-TP, decodifica la respuesta e invoca un callback `on_sample` con el valor y unidades. El periodo de muestreo por defecto es 500 ms. El hilo se inicia y detiene a través de los métodos `start()` y `stop()`, o mediante el protocolo de contexto `with`:

```
with monitor:
    input("Press Enter to stop")
```

Logging SQLite (`sqlite_logger.py`): `SqliteDataLogger` persiste cada muestra en la tabla `samples` (timestamp, PID, valor, unidad y trama hexadecimal) dentro de una base de datos SQLite en modo WAL, que permite escrituras concurrentes desde el hilo de monitorización sin bloqueos.

El CLI incorporó la opción 6 (*Live monitor*) que arranca el monitor y actualiza la pantalla en tiempo real, y la opción 2 (*Extended live data*) que interroga todos los PIDs disponibles en una sola pasada.

Ilustración 5.7: CLI en modo de monitorización continua (opción 6): actualización periódica de los cinco PIDs principales con timestamp, valores decodificados y unidades, ejecutado vía SSH en la Raspberry Pi.

5.2.4. Fase 4: Integración BLE y validación en vehículo real

La fase final añadió el canal de comunicación inalámbrico entre la Raspberry Pi y el teléfono móvil, y trasladó el sistema del banco de pruebas a un vehículo real mediante un cable OBD-II.

Raspberry Pi — servidor BLE GATT

El servidor BLE se implementó en `bluetooth_server.py` mediante la librería `bless`, que expone una interfaz asíncrona (`asyncio`) sobre el stack BlueZ de Linux. El servidor anuncia el **Nordic UART Service (NUS)** con sus dos características estándar:

- ✓ **RX** (0x6E400002-...): característica de escritura. Recibe los comandos JSON del cliente móvil.

- ✓ **TX** (0x6E400003-...): característica de notificación. Envía las respuestas JSON al cliente.

El manejador `_on_write()` acumula los chunks recibidos en un buffer hasta encontrar el delimitador `\n`, deserializa el JSON y despacha el comando al `BtCommandHandler`, que orquesta la sesión de diagnóstico y devuelve el resultado como objeto JSON serializado. Las respuestas que superan el MTU negociado (240 bytes) se fragmentan automáticamente en múltiples notificaciones.

El servidor implementa un mecanismo de *watchdog* que detecta la desconexión del cliente (inactividad superior a 15 s en RX o 20 s en TX) y reinicia el estado de sesión, y un *heartbeat* periódico cada 8 s para mantener el enlace activo durante sesiones de monitorización prolongadas.

Protocolo de comunicación JSON

Los comandos intercambiados entre la aplicación y la Raspberry Pi siguen un protocolo JSON sobre NUS. Los comandos principales son:

- ✓ `{"cmd": "auth", "token": "..."}:` autenticación de sesión (primer comando obligatorio).
- ✓ `{"cmd": "query", "pid": "0C"}:` consulta puntual de un PID del modo 0x01.
- ✓ `{"cmd": "read_dtcs"} / {"cmd": "clear_dtcs"}:` gestión de averías.
- ✓ `{"cmd": "monitor_start", "pids": ["0C", "0D", "05"]}:` inicio de monitorización continua con notificaciones periódicas.
- ✓ `{"cmd": "get_sessions"} / {"cmd": "get_samples", "session_id": N}:` consulta del historial SQLite.

Aplicación móvil — React Native + react-native-ble-plx

La aplicación móvil implementa la comunicación BLE mediante la librería `react-native-ble-plx`, que proporciona acceso al stack BLE nativo de Android e iOS. Las operaciones clave son:

- ✓ **Escaneo y conexión:** `manager.startDeviceScan()` filtra por el nombre del dispositivo (SEAT DIAG) y `device.connect()` establece el enlace físico. Tras la conexión se negocia el MTU, se descubren servicios y se activa la suscripción a notificaciones TX.
- ✓ **Envío de comandos:** `writeCharacteristicWithResponse()` garantiza la entrega del JSON de comando antes de esperar la respuesta.
- ✓ **Recepción de notificaciones:** `monitorCharacteristicForDevice()` invoca un callback con cada fragmento recibido; el adaptador los reasambla antes de procesar el JSON.

Las cinco pantallas de la aplicación se describen en detalle en la Sección 5.6.

Ilustración 5.8: Pantallas *Connection* (izquierda) y *Dashboard* (derecha) de la aplicación móvil durante las pruebas de integración BLE.

Ilustración 5.9: Pantallas *DTCs* (izquierda) y *Logs* (derecha) de la aplicación móvil. La pantalla de DTCs muestra un código inyectado por interrupción hardware; la pantalla de Logs muestra el historial de sesiones almacenado en SQLite.

5.3. Protocolos de comunicación

Esta sección describe los cuatro protocolos de comunicación que conforman la pila de del sistema, desde la capa física del bus CAN hasta el nivel de aplicación . Para cada protocolo se presenta su estructura de trama o paquete, los de control relevantes y la forma en que se ha implementado en este trabajo.

5.3.1. CAN — Controller Area Network (ISO 11898)

El bus CAN es un protocolo de comunicaciones serie orientado a mensajes, diseñado para comunicación entre nodos en entornos con alta interferencia electromagnética. Fue por Robert Bosch GmbH en 1983 y estandarizado bajo la norma ISO 11898.

En su variante CAN 2.0A, que es la empleada en este trabajo, cada trama de datos compuesta por los campos que se recogen en la tabla 5.1.

Cuadro 5.1: Estructura de una trama CAN 2.0A (*Standard Frame*). El campo ID de 11 bits determina la prioridad: a menor valor numérico, mayor prioridad en el bus.

SOF	ID	RTR	IDE	r0	DLC	DATA	CRC	CD	ACK	AD	EOF
1 b	11 b	1 b	1 b	1 b	4 b	0–64 b	15 b	1 b	1 b	1 b	7 b
<i>Arbitration + Control</i>						<i>Datos</i>	<i>CRC</i>		<i>ACK</i>		<i>EOF</i>

El campo **Identificador** (11 bits en CAN 2.0A) determina tanto la prioridad del como su propósito lógico. En el contexto OBD-II, el tester externo emplea el funcional 0x7DF para direccionamiento broadcast o el identificador 0x7E0 para dirigirse directamente a la ECU, mientras que la ECU responde 0x7E8. El mecanismo de arbitraje no destructivo garantiza que, en caso de , el mensaje con identificador de menor valor gana el bus sin que ningún nodo datos.

El campo **DLC** (*Data Length Code*, 4 bits) indica el número de bytes de de la trama, de 0 a 8. OBD-II utiliza tramas de 8 bytes, rellenando con 0x55 0xAA los bytes no utilizados (padding).

En este trabajo, el bus opera a **500 kbps**, velocidad estándar para los sistemas -II modernos. La interfaz **SocketCAN** de Linux, accesible desde la Raspberry Pi través del controlador

MCP2515, expone el bus CAN como un *socket* de red , lo que permite su uso desde Python mediante la librería `python-can` con abstracción de alto nivel.

5.3.2. ISO-TP — Transport Protocol (ISO 15765-2)

El protocolo de transporte ISO-TP define cómo segmentar y reasamblar mensajes de hasta bytes sobre tramas CAN, cuyo campo de datos está limitado a 8 bytes. ISO-TP es capa de transporte obligatoria para la comunicación OBD-II sobre CAN.

ISO-TP define cuatro tipos de trama, identificados por los primeros nibbles del primer de datos (campo PCI, *Protocol Control Information*), tal como se muestra la tabla 5.2.

Cuadro 5.2: Tipos de trama ISO-TP (ISO 15765-2). El byte PCI (en negrita) codifica en sus cuatro bits superiores el tipo de trama y en los inferiores metadatos de longitud o control de flujo. El relleno `0xAA` se usa para completar los 8 bytes del payload CAN.

Tipo	Byte 0	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
SF <i>Single Frame</i>	0x0N	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
FF <i>First Frame</i>	0x1H	0xLL	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆
CF <i>Consecutive Frame</i>	0x2N	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
FC <i>Flow Control</i>	0x3F	BS	STmin	<i>0xAA</i>	<i>0xAA</i>	<i>0xAA</i>	<i>0xAA</i>	<i>0xAA</i>

- N** longitud payload (SF) o número de secuencia (CF, 0x1–0xF)
- H:LL** longitud total del mensaje en 12 bits (FF, máx. 4095 bytes)
- F** flag Flow Control: 0=ContinueSending, 1=Wait, 2=Overflow
- BS** BlockSize: número de CFs antes del siguiente FC (0 = sin límite)
- STmin** separación mínima entre CFs en milisegundos

Para mensajes de hasta 7 bytes (el caso más habitual en OBD-II modo 0x01), se emplea **Single Frame** (SF), que incluye el PCI (**0x0N**, donde N es la longitud), directamente por los bytes de datos.

Para mensajes más largos, como la respuesta al PID **0x09 0x02** (VIN, 17 bytes más longitud), el flujo de transmisión multi-frame ilustrado en la 5.10 sigue la secuencia:

1. El transmisor envía una **First Frame** (FF) con los primeros 6 bytes de datos y la longitud total del mensaje en el campo **FF_DL**.
2. El receptor responde con un **Flow Control** (FC) frame que autoriza la transmisión e indica el tamaño de bloque (**BS**) y el tiempo mínimo entre tramas (**STmin**).
3. El transmisor envía los **Consecutive Frames** (CF) necesarios, con un contador de secuencia (**SN**) de 0 a 15 que se reinicia cíclicamente.

Ilustración 5.10: Flujo de transmisión ISO-TP multi-frame para la respuesta VIN (17 bytes).

La implementación Python utiliza la librería `can-isotp`, que gestiona de forma transparente la segmentación, el Flow Control y la temporización, exponiendo al nivel de aplicación una interfaz de lectura/escritura de mensajes completos. En el simulador Arduino, la secuencia FF+CF se implementa de forma manual sobre la librería CAN del fabricante, con la temporización controlada por `delay()` entre tramas consecutivas.

5.3.3. OBD-II — On-Board Diagnostics (SAE J1979)

OBD-II es el estándar de diagnóstico vehicular que define los servicios y parámetros que toda ECU conforme debe implementar. En la práctica, la comunicación OBD-II se estructura como un protocolo de petición-respuesta a nivel de aplicación, transportado sobre ISO-TP.

La tabla 5.3 ilustra el formato de los mensajes de petición y respuesta para el caso concreto del PID 0x0C (RPM del motor) en modo 0x01.

Cuadro 5.3: Formato de petición y respuesta OBD-II (SAE J1979) encapsulados como payload ISO-TP sobre CAN. Se muestra el ejemplo de lectura de RPM (PID 0x0C).

Petición (Escáner → ECU, CAN ID: 0x7E0)

	PCI	Modo	PID	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
<i>Valor</i>	0x02	0x01	0x0C	0xAA	0xAA	0xAA	0xAA	0xAA
<i>Significado</i>	SF, 2 bytes	Modo 01	RPM	<i>padding</i>				

Respuesta (ECU → Escáner, CAN ID: 0x7E8)

	PCI	Modo	PID	Byte A	Byte B	Byte 5	Byte 6	Byte 7
<i>Valor</i>	0x04	0x41	0x0C	0x1A	0xF0	0xAA	0xAA	0xAA
<i>Significado</i>	SF, 4 bytes	Modo+0x40	RPM	datos RPM		<i>padding</i>		

$$\text{Decodificación: } \text{RPM} = \frac{(A \ll 8) | B}{4} = \frac{0x1AF0}{4} = 1724,0 \text{ rpm}$$

Cada petición OBD-II incluye un byte de modo y, opcionalmente, uno o más bytes de PID. La respuesta de la ECU incluye el modo de respuesta (modo + 0x40), el PID solicitado y los bytes de datos codificados, cuya interpretación depende de la fórmula de decodificación definida por el estándar para cada PID.

El ciclo completo de una consulta OBD-II se ilustra en la figura 5.11, que muestra la propagación del comando desde la aplicación móvil hasta la ECU y el camino de retorno de los datos decodificados.

Los cuatro modos implementados en este trabajo son:

Ilustración 5.11: Ciclo completo de una consulta OBD-II desde la aplicación móvil hasta la ECU.

Ilustración 5.12: Estructura de capas del paquete BLE NUS.

- ✓ **Modo 0x01 — Datos en vivo (PIDs activos).** Permite consultar el valor actual de parámetros operativos del motor. Se han implementado 23 PIDs, incluyendo RPM (0x0C), velocidad (0x0D), temperatura del refrigerante (0x05), presión de combustible (0x0A), posición del acelerador (0x11) y consumo de combustible (0x5E), entre otros.
- ✓ **Modo 0x03 — Lectura de DTCs almacenados.** Devuelve la lista de códigos de avería (*Diagnostic Trouble Codes*) presentes en la memoria de la ECU. Cada DTC se codifica en dos bytes según el formato J2012: el nibble más significativo indica la categoría (P/C/B/U), los 12 bits restantes el código numérico.
- ✓ **Modo 0x04 — Borrado de DTCs.** Borra todos los DTCs almacenados y reinicia el estado del indicador *Check Engine*. Este servicio no requiere PID adicional; la ECU responde con un byte de confirmación (0x44).
- ✓ **Modo 0x09 — Información del vehículo.** Permite obtener el número de identificación del vehículo (VIN, PID 0x02), que en este trabajo se devuelve como una cadena ASCII de 17 caracteres mediante una secuencia ISO-TP multi-frame.

Cuando la ECU no puede atender una petición (modo no soportado, condiciones no adecuadas, PID no implementado), responde con una **Respuesta Negativa UDS** (ISO 14229-1) con identificador 0x7EF. El byte NRC (*Negative Response Code*) identifica la causa del rechazo: 0x11 (servicio no soportado), 0x12 (subfunción no soportada), 0x22 (condiciones no cumplidas) o 0x33 (seguridad no desbloqueada).

5.3.4. BLE GATT y Nordic UART Service

La comunicación inalámbrica entre la Raspberry Pi y la aplicación móvil se implementa mediante Bluetooth Low Energy, usando el perfil GATT (*Generic Attribute Profile*). GATT organiza la comunicación en torno a *servicios* y *características*: cada servicio agrupa un conjunto de características que representan datos o funciones accesibles por el cliente.

El servicio empleado en este trabajo es el **Nordic UART Service (NUS)**, un servicio GATT estándar de facto ampliamente soportado que emula un puerto serie bidireccional sobre BLE. NUS define dos características principales:

- ✓ **TX Characteristic** (6E400003-...): el servidor envía datos al cliente mediante notificaciones GATT. El cliente activa las notificaciones escribiendo en el descriptor CCCD (*Client Characteristic Configuration Descriptor*).
- ✓ **RX Characteristic** (6E400002-...): el cliente escribe datos al servidor mediante operaciones GATT Write.

El MTU negociado durante el establecimiento de la conexión determina el tamaño máximo cada paquete de datos. En este trabajo se configura un MTU de 240 bytes, suficiente para transportar los objetos JSON más grandes de la aplicación (respuestas con múltiples muestras de telemetría) en un único paquete, evitando la fragmentación a nivel de aplicación.

El flujo de establecimiento de conexión se representa en la figura 5.13 e incluye las fases de advertising, conexión, negociación de MTU, descubrimiento de servicios, activación de notificaciones y autenticación por token.

Ilustración 5.13: Flujo de establecimiento de sesión BLE GATT entre la aplicación móvil y la Raspberry Pi.

Sobre la capa de transporte NUS se define un **protocolo de comandos JSON** específico de la aplicación, que estructura los intercambios en forma de petición-respuesta. Cada objeto JSON de petición incluye un campo `cmd` que identifica la operación solicitada, y opcionalmente parámetros adicionales. Las respuestas incluyen siempre un campo `ok` booleano y los datos correspondientes al comando.

5.4. Simulador de ECU — Arduino MKR

El simulador de ECU es el componente que permite validar la herramienta de diagnóstico de forma independiente de un vehículo real. Se implementa sobre un Arduino MKR con shield CAN basado en el controlador MCP2515, conectado al bus CAN a 500 kbps. El código fuente completo del simulador se recoge en el Anexo A.

5.4.1. Hardware y configuración del bus CAN

El Arduino MKR opera a 3,3 V y dispone de comunicación SPI para el controlador MCP2515. La librería CAN utilizada inicializa el controlador a 500 kbps y configura un filtro de aceptación para recibir únicamente los identificadores relevantes: `0x7DF` (broadcast funcional OBD-II) y `0x7E0` (dirección física de la ECU simulada).

Las constantes de configuración del protocolo se concentran en el fichero `ecu_protocol.h`, separando la parametrización del código de lógica. Entre los identificadores definidos destacan:

- ✓ `ECU_CAN_ID = 0x7E0`: identificador del tester (peticiones)
- ✓ `ECU_RESPONSE_ID = 0x7E8`: identificador de respuesta de la ECU
- ✓ `FC_BLOCK_SIZE = 0`: bloqueo de control de flujo (sin límite de bloque)
- ✓ `FC_ST_MIN = 10 ms`: tiempo mínimo entre Consecutive Frames

5.4.2. Modelo físico del motor

Para que los datos de telemetría devueltos por el simulador presenten un comportamiento realista y dinámico, se implementa un modelo físico simplificado del motor que actualiza sus variables de estado en cada iteración del bucle principal.

Las variables del modelo físico son:

- ✓ **RPM:** se modela con una rampa de aceleración/deceleración limitada por la tasa máxima de cambio (`RPM.ACCEL_RATE`). El valor oscila entre el ralentí (~800 rpm) y el máximo configurado, siguiendo una curva pseudoaleatoria que simula el uso normal del vehículo.
- ✓ **Velocidad:** calculada a partir de las RPM mediante la relación de transmisión simulada, con un componente inercial que introduce retardo en la respuesta.
- ✓ **Temperatura del refrigerante:** sigue una curva de calentamiento exponencial desde la temperatura ambiente hasta la temperatura de operación normal (~90 °C), con variaciones pequeñas en estado estacionario.
- ✓ **Temperatura del aceite:** similar a la del refrigerante pero con una constante de tiempo más lenta.
- ✓ **Posición del acelerador:** valor pseudoaleatorio que actúa como entrada del modelo de RPM.
- ✓ **Presión de combustible, tensión de batería, consumo de combustible:** valores derivados de las RPM y la posición del acelerador mediante relaciones lineales con ruido añadido.

El modelo se actualiza con una frecuencia de 100 ms en el bucle principal, y los valores calculados se codifican directamente en los bytes de respuesta OBD-II según las fórmulas del estándar SAE J1979 para cada PID.

5.4.3. Implementación de los modos OBD-II

La función central de decodificación de peticiones `processCANMessage()` analiza el primer byte de datos de cada trama recibida para determinar el modo de servicio, y el segundo byte para el PID específico en el modo 0x01 y 0x09.

5.4.3.1. Modo 0x01 — Datos en vivo

Se implementan 23 PIDs. La respuesta se construye en un buffer de 8 bytes con el formato OBD-II estándar: byte de longitud ISO-TP (tipo SF + longitud), byte de modo de respuesta (0x41), byte de PID y bytes de datos codificados.

La decodificación inversa sigue las fórmulas del estándar para cada PID. Como ejemplo, el PID 0x0C (RPM) codifica el valor en dos bytes según:

$$\text{valor raw} = \text{RPM} \times 4 \quad \Rightarrow \quad \text{byte_A} = \lfloor \text{valor raw} / 256 \rfloor, \quad \text{byte_B} = \text{valor raw} \bmod 256$$

El PID 0x00 (soporte de PIDs 0x01–0x20) devuelve una máscara de bits de 32 bits que indica qué PIDs están disponibles, permitiendo a la herramienta de diagnóstico realizar una enumeración automática de capacidades.

5.4.3.2. Modo 0x03 — Lectura de DTCs

El simulador mantiene una lista de DTCs activos (`dtcList []`) con un contador `numStoredDTCs`. La respuesta del modo 0x03 incluye la cuenta de DTCs seguida de los pares de bytes correspondientes a cada código, codificados según el formato J2012: los dos bits más significativos del primer byte indican la categoría (00=P, 01=C, 10=B, 11=U) y los 12 bits restantes el código numérico en BCD.

5.4.3.3. Modo 0x04 — Borrado de DTCs

Al recibir una petición de modo 0x04, el simulador reinicia `numStoredDTCs` a cero, desactiva el flag `checkEngineLight` y responde con el byte de confirmación 0x44.

5.4.3.4. Modo 0x09 — Información del vehículo (VIN)

El PID 0x02 del modo 0x09 devuelve el VIN del vehículo, una cadena ASCII de 17 caracteres. Dado que la respuesta completa ocupa 20 bytes (3 de cabecera ISO-TP más 17 del VIN), supera la capacidad de una Single Frame y requiere transmisión multi-frame.

La secuencia de transmisión es:

1. **First Frame:** PCI 0x10 0x14 (longitud = 20 bytes), modo 0x49, PID 0x02, contador 0x01 y primeros 3 bytes del VIN.
2. Espera del **Flow Control** del tester.
3. **Consecutive Frame 1:** SN 0x21, bytes 4–10 del VIN.
4. **Consecutive Frame 2:** SN 0x22, bytes 11–17 del VIN (con padding).

Esta implementación manual de la secuencia multi-frame en el simulador fue uno de los puntos más críticos del desarrollo, ya que cualquier error en el contador de secuencia o en el timing entre tramas provocaba el rechazo de la sesión por parte de la librería `can-isotp` en la Raspberry Pi.

5.4.4. Mecanismos de simulación de ruido CAN

Una de las aportaciones más relevantes del simulador es la capacidad de reproducir condiciones adversas del bus CAN de forma controlada y reproducible. Los mecanismos de ruido se activan mediante el flag `canNoiseEnabled` y se aplican durante el procesamiento de cada petición recibida.

5.4.4.1. Latencia variable

Antes de procesar la petición, el simulador introduce un retardo aleatorio entre `CAN_LATENCY_MIN_MS` (1 ms) y `CAN_LATENCY_MAX_MS` (15 ms) mediante `delay(random(...))`. Este retardo simula la variabilidad de la carga del bus en un entorno vehicular real, donde múltiples ECUs compiten por el acceso al medio de forma simultánea.

5.4.4.2. Descarte de paquetes

Con una probabilidad configurable (`CAN_DROP_PCT = 2%`), el simulador descarta la petición sin emitir respuesta alguna, simplemente retornando de la función de procesamiento. Este comportamiento simula la pérdida de tramas por colisión o por degradación de la señal en el bus.

5.4.4.3. Respuestas negativas esporádicas

Con una probabilidad independiente (`CAN_NRC_PCT = 1%`), el simulador responde con una respuesta negativa UDS con NRC `0x22` (*conditionsNotCorrect*), que indica que la ECU no puede atender la petición en ese momento. Esta simulación reproduce el comportamiento real de las ECUs cuando reciben peticiones fuera de su ventana de ejecución.

5.4.4.4. Tráfico de fondo

La función `generateBackgroundTraffic()` se invoca en cada iteración del bucle principal y, con una probabilidad del 30 % por ciclo, envía una trama CAN con un identificador aleatorio de entre cuatro seleccionados (`0x280`, `0x480`, `0x320`, `0x520`) y 8 bytes de datos aleatorios. Este tráfico imita las transmisiones periódicas de otras ECUs del vehículo (módulo de carrocería, ABS, dirección asistida) y permite verificar que el filtro de aceptación de la Raspberry Pi funciona correctamente bajo carga.

5.4.5. Inyección de DTCs por interrupción hardware

Para simular la aparición espontánea de averías durante una sesión de diagnóstico, el simulador implementa un mecanismo de inyección de DTCs activado por la señal en el pin digital

D6 (`DTC_FAULT_PIN`).

El patrón de implementación sigue la práctica estándar para ISR en Arduino:

1. La ISR `onDtcFaultButton()` es minimalista: únicamente activa un flag `volatile bool dtcFaultPressed = true`.
2. La lógica de debounce (300 ms) y la inyección propiamente dicha se realizan en la función `handleDtcFaultButton()`, llamada desde el bucle principal, donde es seguro ejecutar operaciones con efectos de bloqueo.
3. Se selecciona aleatoriamente uno de los 12 DTCs disponibles en la tabla del simulador: P0300, P0301, P0302, P0303, P0113, P0118, P0340, P0341, P0500, P0501, P0171, P0172.
4. Se comprueba que el código seleccionado no esté ya presente en `dtcList[]` antes de insertarlo (deduplicación), y se activa el flag `checkEngineLight`.

Esta funcionalidad permite al operador del banco de pruebas introducir averías de forma controlada durante una sesión de diagnóstico activa y verificar que la herramienta Python las detecta correctamente en la siguiente consulta del modo 0x03.

5.5. Herramienta de diagnóstico — Raspberry Pi

La herramienta de diagnóstico es el componente central del sistema: recibe peticiones desde la aplicación móvil vía BLE, las traduce a secuencias ISO-TP sobre el bus CAN, interpreta las respuestas de la ECU, persiste los datos en SQLite y devuelve los resultados al cliente móvil. Se implementa en Python 3 sobre Raspberry Pi con controlador CAN MCP2515. El código fuente se recoge en el Anexo B.

5.5.1. Arquitectura software

La herramienta sigue un diseño basado en interfaces y capas desacopladas mediante inyección de dependencias. Esta decisión arquitectónica, que supuso una inversión inicial de tiempo mayor a la contemplada en el diseño rápido, resultó rentable en las fases posteriores al facilitar las pruebas sin hardware y la extensión progresiva del sistema.

Las interfaces abstractas principales son:

- ✓ **ITransport**: define las operaciones de envío y recepción de mensajes ISO-TP (`send`, `recv`). Las implementaciones concretas son `CanTransport` (bus CAN real mediante `can-isotp`) y `MockTransport` (respuestas predefinidas para pruebas sin hardware).
- ✓ **IProtocolBuilder**: define la construcción de mensajes OBD-II (`build_request`). La implementación `OBD2ProtocolBuilder` genera las tramas de petición según el estándar SAE J1979.

- ✓ **IDataDecoder**: define la decodificación de respuestas OBD-II (`decode_response`). La implementación `OBD2DataDecoder` aplica las fórmulas del estándar para cada PID y devuelve valores con unidades.
- ✓ **IDiagnosticSession**: define la interfaz de sesión de diagnóstico (`query_pid`, `read_dtcs`, `clear_dtcs`, `get_vin`). La implementación `DiagnosticSession` orquesta las demás interfaces.

Sobre esta estructura base se aplica el patrón decorador mediante **LoggingTransport**, que envuelve cualquier `ITransport` e intercepta todas las operaciones de envío y recepción para registrarlas en la base de datos SQLite, sin modificar el comportamiento del transporte subyacente. La composición resultante es:

```
LoggedDiagnosticSession(LoggingTransport(CanTransport), OBD2ProtocolBuilder, OBD2DataDecoder)
```

5.5.2. Capa de transporte CAN e ISO-TP

La comunicación con el bus CAN se realiza a través de la interfaz `SocketCAN` de Linux, configurada mediante `ip link set can0 up type can bitrate 500000`. La librería `python-can` proporciona una abstracción de alto nivel sobre esta interfaz, mientras que `can-isotp` gestiona de forma automática la segmentación y reensamblado ISO-TP, el control de flujo y la temporización entre tramas.

La configuración del socket ISO-TP define la dirección del tester (`0x7E0`) y la dirección de respuesta de la ECU (`0x7E8`), con `padding` activado para cumplir con el relleno a 8 bytes requerido por el estándar. El timeout de recepción se fija en 1 segundo, valor suficiente para absorber la latencia máxima introducida por el simulador (15 ms) con amplio margen.

5.5.3. Módulo de logging en SQLite

Toda la información de diagnóstico se persiste en una base de datos SQLite cuyo esquema se detalla en el Anexo D. La base de datos opera en modo WAL (*Write-Ahead Logging*), que permite escrituras concurrentes desde el hilo de monitorización y el hilo de atención a comandos BLE sin bloqueos mutuos.

Las tres tablas principales del esquema son:

- ✓ **sessions**: registra cada sesión de diagnóstico con su timestamp de inicio y fin, el identificador del dispositivo cliente BLE y el número de muestras recogidas.
- ✓ **samples**: almacena cada muestra de telemetría con el timestamp, el PID consultado, el valor decodificado, la unidad y la trama CAN en hexadecimal (campo `raw_hex`). Este último permite reconstruir la secuencia completa de comunicación ISO-TP para análisis forense.

- ✓ **commands**: registra cada comando ejecutado desde la aplicación móvil con el timestamp, el tipo de comando, la dirección BLE del cliente y el resultado.

El `LoggingTransport` acumula las muestras en un buffer en memoria y las vuelca a SQLite en lotes de 50, reduciendo la frecuencia de escritura a disco y mejorando el rendimiento general del sistema durante sesiones de monitorización prolongadas.

5.5.4. Monitor de datos en tiempo real

El hilo de monitorización continua (`MonitorThread`) se ejecuta en segundo plano con un periodo de muestreo de 500 ms. En cada ciclo consulta un conjunto configurable de PIDs del modo 0x01, decodifica los valores recibidos y los publica en una cola compartida (`queue.Queue`) que el hilo BLE lee para enviar notificaciones al cliente móvil.

El conjunto de PIDs monitorizados por defecto incluye RPM, velocidad, temperatura del refrigerante, posición del acelerador y carga calculada del motor. La lista es configurable en tiempo de ejecución a través del comando JSON correspondiente.

5.5.5. Servidor BLE GATT

El servidor BLE se implementa mediante la librería `bless`, que proporciona una interfaz asíncrona (`asyncio`) para la gestión del servicio GATT. El servidor expone el servicio NUS con sus dos características (TX y RX) y gestiona las conexiones de los clientes BLE de forma concurrente.

El manejador de comandos `BtCommandHandler` recibe los objetos JSON escritos por el cliente en la característica RX, los despacha al método correspondiente de la `LoggedDiagnosticSession` y envía la respuesta JSON como notificación en la característica TX.

Se implementa un mecanismo de autenticación por token: el primer comando de cada sesión BLE debe incluir el campo `token` con el valor configurado en el servidor. Las conexiones no autenticadas reciben una respuesta de error y se cierran tras un timeout de inactividad configurable.

Los comandos JSON implementados son:

- ✓ `{"cmd": "auth", "token": "..."}:` autenticación de sesión.
- ✓ `{"cmd": "query", "pid": "0C"}:` consulta un PID del modo 0x01.
- ✓ `{"cmd": "read_dtcs"}:` lectura de los DTCs almacenados.
- ✓ `{"cmd": "clear_dtcs"}:` borrado de DTCs.
- ✓ `{"cmd": "get_vin"}:` obtención del VIN del vehículo.
- ✓ `{"cmd": "monitor_start", "pids": ["0C","0D","05"]}:` inicio de monitorización continua.

- ✓ {"cmd": "monitor_stop"}: detención de la monitorización.
- ✓ {"cmd": "get_sessions"}: lista de sesiones históricas en SQLite.
- ✓ {"cmd": "get_samples", "session_id": N}: muestras de una sesión.

5.6. Aplicación móvil — React Native

La aplicación móvil es la interfaz de usuario del sistema de diagnóstico. Se implementa con React Native y Expo, lo que permite compartir una única base de código entre Android e iOS. El código fuente se recoge en el Anexo C.

5.6.1. Arquitectura de la aplicación

La aplicación sigue una arquitectura basada en tres capas:

1. **Capa de adaptadores:** abstrae el acceso al hardware BLE mediante la interfaz `IBleAdapter`. La implementación concreta `BleAdapter` utiliza la librería `react-native-ble-plx` para las operaciones de escaneo, conexión, escritura y suscripción a notificaciones. La implementación `MockAdapter` simula un dispositivo BLE para el desarrollo sin hardware, con respuestas predefinidas para todos los comandos.
2. **Capa de estado:** gestiona el estado global de la aplicación mediante almacenes Zustand. Los almacenes principales son `useConnectionStore` (estado de la conexión BLE y dispositivo activo), `useTelemetryStore` (últimos valores de los PIDs monitorizados), `useDtcStore` (lista de DTCs activos) y `useSessionStore` (historial de sesiones y muestras).
3. **Capa de presentación:** las cinco pantallas de la aplicación consumen el estado de los almacenes Zustand y despachan comandos a través del adaptador BLE.

5.6.2. Pantallas y funcionalidades

5.6.2.1. Pantalla Connection

La pantalla de conexión permite al usuario escanear dispositivos BLE cercanos, filtrar por nombre (`PiDiag`) y establecer la conexión con la Raspberry Pi. Durante el proceso de conexión se muestra el progreso de las fases: escaneo, conexión física, negociación de MTU, descubrimiento de servicios, suscripción a notificaciones y autenticación por token.

El adaptador BLE implementa reintentos automáticos con *backoff* exponencial ante fallos de conexión, con un máximo de tres intentos antes de notificar el error al usuario. Un timeout configurable de inactividad BLE desconecta automáticamente la sesión si no hay actividad durante el periodo establecido.

5.6.2.2. Pantalla Dashboard

La pantalla principal del diagnóstico muestra en tiempo real los parámetros del vehículo recibidos por monitorización continua. Los indicadores se presentan en forma de tarjetas con el valor actual, la unidad y una barra de progreso visual que refleja el rango normal de operación de cada parámetro.

Al establecer la conexión, la aplicación inicia automáticamente la monitorización de un conjunto por defecto de PIDs: RPM, velocidad, temperatura del refrigerante y posición del acelerador. El usuario puede añadir o eliminar PIDs de la lista de monitorización activa.

5.6.2.3. Pantalla DTCs

La pantalla de códigos de avería muestra la lista de DTCs activos leída desde la ECU. Cada código se presenta con su identificador alfanumérico (formato J2012), su descripción en texto y un indicador de categoría (P/C/B/U). La pantalla ofrece la acción de borrado de todos los DTCs, con un diálogo de confirmación antes de ejecutar el comando al ser una operación irreversible.

5.6.2.4. Pantalla Logs

La pantalla de historial muestra la lista de sesiones de diagnóstico almacenadas en la base de datos SQLite de la Raspberry Pi, ordenadas por fecha. Al seleccionar una sesión, se presenta la lista de muestras de telemetría con sus valores y timestamps. Cada muestra puede expandirse para mostrar la trama CAN en hexadecimal correspondiente, permitiendo la inspección detallada de la comunicación a nivel de protocolo.

5.6.2.5. Pantalla Console

La consola de comandos directa permite al usuario enviar comandos JSON arbitrarios a la Raspberry Pi y visualizar la respuesta en formato JSON formateado. Esta funcionalidad está orientada al desarrollo y la depuración, y permite probar comandos fuera del flujo habitual de la interfaz gráfica.

5.6.3. Comunicación BLE

La librería `react-native-ble-plx` gestiona las operaciones BLE de bajo nivel. La característica RX del servicio NUS se escribe mediante operaciones `writeCharacteristicWithResponse`, que garantizan la entrega del comando antes de esperar la respuesta. La característica TX se monitoriza mediante `monitorCharacteristicForDevice`, que invoca un callback cada vez que la Raspberry Pi envía una notificación.

Los objetos JSON de respuesta que superan el MTU negociado se fragmentan a nivel de aplicación: la Raspberry Pi los divide en fragmentos con un campo `chunky` los reasambla la aplicación antes de procesarlos. En la práctica, con el MTU de 240 bytes configurado, este caso solo se produce en las respuestas de historial de sesiones con un número elevado de muestras.

5.7. Integración y pruebas

La validación del sistema completo se realizó sobre el banco de ensayo formado por el Arduino MKR con shield CAN, la Raspberry Pi con controlador MCP2515 y un teléfono Android con la aplicación instalada. Los tres nodos se conectaron mediante un bus CAN de dos hilos de 30 cm de longitud con terminación de 120Ω en cada extremo.

5.7.1. Escenarios de prueba

Se definieron los siguientes escenarios de prueba que cubren los flujos principales del sistema y los casos de error más relevantes:

Escenario 1 — Ciclo OBD-II básico. Verificación del ciclo completo de consulta de un PID del modo 0x01 (RPM, 0x0C): envío de la petición desde la aplicación móvil, transmisión CAN de la petición ISO-TP, respuesta del simulador con el valor codificado, decodificación en la Raspberry Pi y presentación en el Dashboard. Se verificó la coherencia del valor decodificado con el modelo físico del simulador y la correcta persistencia de la muestra en SQLite.

Escenario 2 — Respuesta VIN multi-frame. Verificación de la secuencia ISO-TP completa para la consulta del VIN (0x09 0x02): First Frame, Flow Control y dos Consecutive Frames. Se comprobó que el VIN reconstruido coincide exactamente con la cadena configurada en el simulador y que las tres tramas CAN correspondientes se registran individualmente en la base de datos.

Escenario 3 — Gestión de DTCs. Verificación del ciclo completo de gestión de averías: lectura de DTCs inicial (lista vacía), inyección de un DTC mediante la interrupción hardware del pin D6, lectura de DTCs tras la inyección (DTC presente), borrado de DTCs mediante la aplicación y lectura final (lista vacía de nuevo).

Escenario 4 — Monitorización continua prolongada. Sesión de monitorización de 10 minutos con cuatro PIDs activos (RPM, velocidad, temperatura, acelerador) a 500 ms de periodo. Se verificó la estabilidad de la conexión BLE, la ausencia de pérdidas de muestras y el correcto comportamiento del sistema bajo carga sostenida.

Escenario 5 — Robustez ante ruido CAN. Verificación del comportamiento de la herramienta Python ante los mecanismos de ruido del simulador activos. Se comprobó que el sistema recupera correctamente de los descartes de paquetes (retransmisión ISO-TP automática), que las respuestas negativas UDS se registran en el log y no bloquean el hilo de

monitorización, y que el tráfico de fondo no es interpretado erróneamente como respuesta válida.

Escenario 6 — Reconexión BLE. Verificación del mecanismo de reconexión automática tras la pérdida de la conexión Bluetooth: el adaptador BLE detecta la desconexión, inicia el proceso de reintento con backoff exponencial y reanuda la sesión de monitorización sin pérdida de datos históricos.

5.7.2. Resultados de las pruebas de integración

Todos los escenarios de prueba definidos se completaron con resultado satisfactorio en el banco de ensayo. Los aspectos más relevantes observados durante las pruebas se describen en detalle en el Capítulo 6.

La principal dificultad encontrada durante las pruebas de integración fue la sincronización de la secuencia ISO-TP multi-frame para el VIN. En las primeras iteraciones, el simulador Arduino enviaba el Flow Control con un valor de `STmin` incorrecto que provocaba que la librería `can-isotp` de la Raspberry Pi rechazara la sesión por timeout. La corrección consistió en reducir `FC_ST_MIN` a 10 ms y añadir un delay explícito entre la recepción del Flow Control y el envío del primer Consecutive Frame en el simulador.

Las pruebas con ruido CAN confirmaron que el mecanismo de descarte aleatorio de paquetes (2% de probabilidad) requería un timeout de recepción suficientemente generoso en la herramienta Python para no generar falsos errores en condiciones de baja carga del bus. Con el timeout de 1 segundo finalmente establecido, la tasa de errores no recuperables se situó por debajo del 0,1 % en todas las sesiones de prueba.

Capítulo 6

Resultados

6.1. Resultados obtenidos

Este capítulo presenta los resultados de la validación del sistema en dos entornos complementarios: el banco de ensayo Arduino, que permitió reproducir condiciones adversas de forma controlada, y un vehículo real, en el que se verificó la interoperabilidad del sistema con una ECU de producción a través del conector OBD-II estándar. Los resultados se organizan en torno a cuatro dimensiones: funcionalidad del protocolo, rendimiento de la comunicación, comportamiento ante condiciones adversas y grado de consecución de los objetivos planteados en el Capítulo 2.

6.2. Validación sobre vehículo real

Tras completar la validación sobre el banco de ensayo, el sistema se probó sobre un vehículo real utilizando un cable OBD-II estándar conectado al puerto de diagnóstico del vehículo. La Raspberry Pi se sustituyó en este escenario como interfaz entre el bus CAN del vehículo y la aplicación móvil, manteniendo exactamente el mismo stack de software validado en el banco: `IsoTpTransport` sobre `SocketCAN`, servidor BLE GATT con NUS y aplicación `React Native`.

La conexión al vehículo demostró que el protocolo OBD-II implementado es compatible con ECUs de producción: la herramienta fue capaz de consultar los PIDs del modo 0x01 soportados por el vehículo, leer el VIN real mediante la secuencia ISO-TP multi-frame y recibir datos de telemetría en tiempo real a través de la aplicación móvil. Las respuestas del vehículo presentaron tiempos de respuesta ligeramente superiores a los del simulador (10–30 ms frente a 2–8 ms), dentro del margen absorbido por el timeout configurado.

Ilustración 6.1: Sistema montado en el vehículo real: cable OBD-II conectado al puerto de diagnóstico, Raspberry Pi con el servidor activo y teléfono móvil con la aplicación mostrando datos de telemetría en tiempo real.

6.3. Validación funcional del protocolo OBD-II

La herramienta de diagnóstico fue capaz de completar satisfactoriamente el ciclo completo de consulta para los 23 PIDs implementados en el modo 0x01, los cuatro servicios del modo 0x09 (incluyendo el VIN multi-frame) y los modos de gestión de DTCs 0x03 y 0x04.

La tabla 6.1 resume los parámetros validados más representativos, indicando el PID, la fórmula de decodificación aplicada y el rango de valores observados durante las sesiones de prueba.

Cuadro 6.1: PIDs del modo 0x01 validados en el banco de ensayo.

PID	Parámetro	Fórmula (A, B = bytes de respuesta)	Rango observado
0x0C	RPM del motor	$((A \times 256) + B)/4$	800 – 6000 rpm
0x0D	Velocidad	A	0 – 220 km/h
0x05	Temp. refrigerante	$A - 40$	15 – 95 °C
0x0F	Temp. aire admisión	$A - 40$	15 – 45 °C
0x0A	Presión combustible	$A \times 3$	240 – 360 kPa
0x11	Posición acelerador	$A \times 100/255$	0 – 100 %
0x04	Carga calculada motor	$A \times 100/255$	10 – 85 %
0x5E	Consumo combustible	$((A \times 256) + B)/20$	0,4 – 8,5 L/h
0x42	Tensión de batería	$((A \times 256) + B)/1000$	12,4 – 14,8 V

La respuesta VIN se validó con una cadena de 17 caracteres ASCII, verificando que la secuencia ISO-TP multi-frame (FF + FC + CF $\times$ 2) se completaba correctamente en el 100 % de los intentos realizados sin ruido CAN activo. El VIN reconstruido en la herramienta Python coincidió en todos los casos con la cadena configurada en el simulador Arduino.

El ciclo de gestión de DTCs se validó íntegramente: lectura de lista vacía, inyección de tres DTCs distintos mediante la interrupción hardware del pin D6, lectura de la lista con los tres códigos presentes, borrado y confirmación de lista vacía. La codificación J2012 de los DTCs fue correcta en todos los casos.

6.4. Métricas de rendimiento de la comunicación

Se midió la latencia extremo a extremo de una consulta OBD-II completa, desde el envío del comando JSON desde la aplicación móvil hasta la recepción de la respuesta decodificada, en

condiciones de bus sin ruido. Las mediciones se realizaron sobre 100 consultas consecutivas del PID 0x0C (RPM).

Los resultados obtenidos se resumen en la tabla 6.2:

Cuadro 6.2: Latencia extremo a extremo de una consulta OBD-II (100 muestras, sin ruido CAN).

Componente	Latencia media (ms)
BLE Write (app → Raspberry Pi)	12 – 25
Procesamiento JSON + construcción trama	< 1
Transacción ISO-TP CAN (SF)	2 – 8
Decodificación + logging SQLite	3 – 7
BLE Notify (Raspberry Pi → app)	10 – 20
Total extremo a extremo	28 – 62

La latencia dominante en el sistema es la introducida por la capa BLE, que suma las contribuciones del Write y la Notify. La latencia CAN+ISO-TP es despreciable frente a la BLE a las velocidades de bus utilizadas.

En el modo de monitorización continua con cuatro PIDs a 500 ms de periodo, el sistema fue capaz de mantener una tasa de actualización estable sin pérdidas de muestras durante sesiones de 10 minutos. La cola de datos entre el hilo de monitorización y el hilo BLE no presentó saturación en ninguno de los ensayos.

La base de datos SQLite acumuló aproximadamente 1.200 muestras por sesión de 10 minutos con cuatro PIDs, con un tamaño de fichero resultante de 480 KB. La escritura en lotes de 50 muestras mantuvo la latencia de acceso a disco por debajo de 15 ms en el 99 % de los casos.

6.5. Comportamiento ante condiciones adversas

Se evaluó el comportamiento del sistema con los mecanismos de ruido CAN del simulador activos simultáneamente: latencia variable (1–15 ms), descarte de paquetes (2 %), respuestas negativas UDS esporádicas (1 %) y tráfico de fondo (30 % por ciclo).

Los resultados más relevantes de las pruebas bajo ruido son:

- ✓ **Descarte de paquetes:** La librería `can-isotp` gestionó los descartes de forma transparente gracias al mecanismo de retransmisión por timeout. Con el timeout de 1 segundo configurado, el 100 % de los mensajes descartados fueron recuperados en el primer o segundo reintento, sin que ningún descarte provocara un error fatal en la sesión.
- ✓ **Respuestas negativas UDS:** Las respuestas NRC 0x22 fueron detectadas y registradas correctamente en el log de comandos SQLite. El hilo de monitorización continuó

omite la muestra correspondiente y prosigue con el siguiente PID del ciclo, sin bloquear la sesión.

- ✓ **Tráfico de fondo:** Las tramas de tráfico de fondo (identificadores 0x280, 0x480, 0x320, 0x520) fueron ignoradas por el filtro de aceptación configurado en el socket `can-isotp`, sin interferencia alguna en la comunicación OBD-II.
- ✓ **Latencia variable:** El incremento de latencia hasta 15ms en la capa CAN resultó imperceptible a nivel de aplicación, dado que la latencia dominante del sistema es la capa BLE (>10ms). La monitorización continua no registró pérdidas de muestras atribuibles a la latencia CAN.

La tasa de errores no recuperables (timeouts que provocan el descarte definitivo de una muestra) se situó en un 0,08 % sobre el total de consultas realizadas con ruido activo, valor que se considera aceptable para el uso previsto del sistema.

6.6. Grado de consecución de los objetivos

La tabla 6.3 recoge el grado de consecución de cada uno de los objetivos específicos definidos en la Sección 2.2, evaluado sobre la base de los resultados de las pruebas de integración.

Cuadro 6.3: Evaluación del grado de consecución de los objetivos específicos.

Obj.	Indicador de consecución	Resultado
1	Pila ISO-TP + OBD-II modos 0x01/03/04/09 operativa	Completo
2	Simulador Arduino responde a los 4 modos con datos realistas	Completo
3	Ruido CAN: descarte, NRC, latencia, tráfico de fondo	Completo
4	Servidor BLE GATT con NUS operativo en Raspberry Pi	Completo
5	App React Native Android/iOS con 5 pantallas funcionales	Completo
6	Logging SQLite con sesiones, muestras y comandos	Completo

Los seis objetivos específicos se han alcanzado en su totalidad. La validación sobre vehículo real (Sección 6.2) confirmó la compatibilidad del sistema con ECUs de producción, completando la cadena de validación desde el banco de ensayo controlado hasta un entorno vehicular real.

Capítulo 7

Conclusiones y trabajo futuro

7.1. Conclusiones

Este Trabajo de Fin de Título ha demostrado la viabilidad técnica de implementar un sistema completo de diagnóstico vehicular OBD-II sobre hardware de bajo coste, sin recurrir a adaptadores comerciales que abstraen el protocolo mediante firmware cerrado. Los resultados obtenidos en el banco de ensayo Arduino confirman que la plataforma desarrollada es funcional, robusta ante condiciones adversas del bus CAN y extensible en sus tres componentes de forma independiente.

Las conclusiones técnicas más relevantes del trabajo se agrupan en torno a los tres componentes del sistema:

Sobre la pila de protocolos. La implementación directa de ISO-TP y OBD-II sobre `SocketCAN` en Python ha resultado viable y de rendimiento adecuado para el uso previsto. La librería `can-isotp` proporciona una abstracción suficiente para gestionar la segmentación y el Flow Control de forma transparente, si bien la correcta sincronización de la secuencia multi-frame para el VIN requirió un ajuste cuidadoso de los parámetros de timing en el simulador. La latencia extremo a extremo del sistema (28–62ms por consulta) es significativamente inferior a la introducida por los adaptadores ELM327, cuyo protocolo AT añade retardos de cientos de milisegundos.

Sobre el simulador Arduino. El simulador de ECU desarrollado se ha revelado como una herramienta de validación de alto valor, que va más allá de la simple emulación de respuestas fijas. El modelo físico del motor, los mecanismos de ruido CAN controlado y la inyección de DTCs por interrupción hardware dotan al banco de ensayo de una capacidad de simulación de escenarios reales que resulta difícil de reproducir sin acceso continuo a un vehículo. Esta combinación de funcionalidades no se encuentra, al menos de forma integrada, en las implementaciones similares disponibles en la literatura revisada.

Sobre la arquitectura software. La decisión de estructurar la herramienta Python con interfaces abstractas e inyección de dependencias ha demostrado su valor a lo largo del desa-

rrollo: la posibilidad de sustituir el transporte CAN real por un *mock* en cualquier momento redujo significativamente los ciclos de depuración y permitió trabajar con la capa de aplicación de forma independiente del hardware. Esta arquitectura se considera una contribución metodológica de este trabajo, reproducible en proyectos de naturaleza similar.

Desde el punto de vista del proceso de desarrollo, la metodología basada en prototipos incrementales resultó especialmente adecuada para un proyecto de esta naturaleza, en el que la validez de las hipótesis técnicas no podía garantizarse a priori. Los ciclos cortos de implementación y prueba sobre el banco de ensayo permitieron detectar y corregir problemas de sincronización ISO-TP en etapas tempranas, antes de que estos afectaran al desarrollo de las capas superiores.

Como limitación principal del trabajo cabe señalar la ausencia de validación sobre vehículo real, que hubiera permitido confirmar el comportamiento del sistema frente a la arquitectura Gateway Central presente en el SEAT Ibiza objetivo. Esta limitación, debida a restricciones de acceso al hardware vehicular durante el período de desarrollo, no invalida los resultados obtenidos en el banco de ensayo, pero sí condiciona el alcance de las conclusiones sobre la aplicabilidad práctica del sistema.

7.2. Líneas de trabajo futuro

Los resultados obtenidos y las limitaciones identificadas abren varias líneas de continuación natural para este trabajo:

Validación sobre vehículo real. La línea más inmediata es la prueba del sistema sobre el vehículo objetivo (SEAT Ibiza PQ25 o similar con arquitectura Gateway Central). Esta validación permitiría verificar si la herramienta es capaz de comunicarse correctamente a través del Gateway J533, identificar las diferencias entre el comportamiento del simulador y la ECU real, y calibrar el modelo físico del simulador con datos reales.

Soporte de servicios UDS extendidos. El sistema actual se limita a los modos OBD-II estándar definidos por SAE J1979. Una extensión natural es el soporte de servicios UDS adicionales (ISO 14229-1), como la lectura de datos por identificador (0x22), la escritura de datos (0x2E) o el control de rutinas (0x31), que permiten acceder a parámetros de configuración y calibración de las ECUs no expuestos por el estándar OBD-II.

Soporte de CAN FD. El estándar CAN FD (ISO 11898-7), ampliamente adoptado en los vehículos más modernos, permite tramas de hasta 64 bytes y velocidades de datos de hasta 8 Mbps. La migración del sistema a CAN FD requeriría la sustitución del controlador MCP2515 por un controlador compatible con CAN FD, así como la adaptación de la capa ISO-TP (ISO 15765-4 define las extensiones para CAN FD).

Visualización gráfica de tendencias. La aplicación móvil actual presenta los valores de telemetría como indicadores numéricos instantáneos. Una mejora de alto valor para el usuario sería la incorporación de gráficas de tendencia temporal que permitan observar la evolución

de los parámetros durante la sesión, funcionalidad que fue descartada del alcance inicial por restricciones de tiempo.

Exportación de datos y análisis offline. La base de datos SQLite de la Raspberry Pi almacena toda la información de la sesión, incluyendo las tramas CAN en hexadecimal. Una extensión útil sería la implementación de un módulo de exportación a formato CSV o JSON que permita analizar los datos offline con herramientas como pandas o importarlos en plataformas de visualización de datos.

Seguridad en la comunicación BLE. El mecanismo de autenticación por token implementado proporciona un nivel básico de protección, pero no cifra el canal de comunicación. Para un uso en entornos de producción sería recomendable implementar cifrado de la comunicación BLE mediante los mecanismos definidos en la especificación Bluetooth 5.x o bien añadir un protocolo de seguridad a nivel de aplicación.

7.3. Reflexión sobre el uso de herramientas de inteligencia artificial

Durante el desarrollo de este trabajo se hizo uso de herramientas de inteligencia artificial generativa como asistente de programación, concretamente mediante el acceso a modelos de lenguaje de gran escala a través de interfaces de línea de comandos.

El uso de estas herramientas se circunscribió a tres contextos concretos: la generación de fragmentos de código repetitivos o bien definidos por un estándar (como las tablas de decodificación de PIDs OBD-II), la resolución de dudas puntuales sobre la API de las librerías utilizadas (`python-can`, `bless`, `react-native-ble-plx`) y la revisión de la sintaxis de LaTeX en secciones con tablas complejas.

En ningún caso se delegó en estas herramientas el diseño de la arquitectura del sistema, la implementación de los protocolos de comunicación ni la interpretación de los resultados experimentales. Todas las decisiones técnicas relevantes, el análisis de los errores encontrados durante las pruebas de integración y la redacción de las conclusiones son producto del trabajo propio del autor.

La experiencia ha puesto de manifiesto tanto el valor como los límites de estas herramientas en el contexto de un proyecto de ingeniería. Son especialmente útiles para reducir el tiempo invertido en tareas de codificación mecánica y para obtener una primera orientación ante problemas con librerías poco documentadas. Sin embargo, la generación de código incorrecto sin diagnóstico claro del error sigue siendo habitual, y la verificación de cada sugerencia mediante prueba real es indispensable. El criterio y el conocimiento del dominio del ingeniero siguen siendo el elemento central e insustituible del proceso de desarrollo.

Anexo A — Código fuente del simulador Arduino

Este anexo recoge los fragmentos más representativos del código fuente del simulador de ECU implementado sobre Arduino MKR. El código completo está disponible en el repositorio del proyecto.

A.1 — Definiciones de protocolo (ecu_protocol.h)

El fichero de cabecera centraliza todas las constantes del protocolo, las estructuras de datos y las funciones de codificación/decodificación OBD-II.

```
1 // ----- CAN BUS -----
2 #define CAN_SPEED          500E3
3 #define ECU_CAN_ID         0x7E0
4 #define ECU_RESPONSE_ID    0x7E8
5
6 // ----- ISO-TP -----
7 #define ISOTP_PCI_SF        0x00 // Single Frame
8 #define ISOTP_PCI_FF        0x10 // First Frame
9 #define ISOTP_PCI_CF        0x20 // Consecutive Frame
10 #define ISOTP_PCI_FC        0x30 // Flow Control
11
12 #define ISOTP_CF_SEP_MS      25
13 #define ISOTP_SF_MAX_PAYLOAD 7
14 #define ISOTP_FF_DATA_BYTES 6
15
16 // ----- CAN NOISE -----
17 #define CAN_NOISE_DROP_PCT   2
18 #define CAN_NOISE_NRC_PCT    1
19 #define CAN_NOISE_LATENCY_MIN_MS 1
20 #define CAN_NOISE_LATENCY_MAX_MS 15
21 #define CAN_NOISE_BG_TRAFFIC_PCT 30
22 static const uint32_t CAN_BG_IDS[4] = {0x280, 0x480, 0x320, 0x520};
23
24 // ----- HARDWARE -----
25 #define DTC_FAULT_PIN        6
26 #define DTC_DEBOUNCE_MS      300
```

Listado 1: Constantes de CAN, ISO-TP y ruido — ecu_protocol.h

```

1 struct VehicleData {
2     char    vin[18];
3     uint16_t rpm;
4     uint8_t  speed;
5     int16_t  coolantTemp;
6     uint8_t  throttlePos;
7     uint8_t  engineLoad;
8     uint16_t batteryVoltage; // mV
9     bool     checkEngine;
10    uint8_t  numDTCs;
11 };
12
13 // Codificación OBD-II: rpm*4 en dos bytes big-endian
14 inline void encodeRPM(uint8_t* data, uint16_t rpm) {
15     uint16_t encoded = rpm * 4;
16     data[0] = (encoded >> 8) & 0xFF;
17     data[1] = encoded & 0xFF;
18 }
19
20 // Temperatura: T_raw = T_celsius + 40
21 inline uint8_t encodeTemp(int16_t temp) {
22     return (uint8_t)(temp + 40);
23 }
24
25 // Porcentaje: val_raw = percent * 255 / 100
26 inline uint8_t encodePercent(uint8_t percent) {
27     return (percent * 255) / 100;
28 }

```

Listado 2: Estructura VehicleData y funciones de codificación OBD-II

A.2 — Bucle principal y mecanismos de ruido CAN (ecu_sim.ino)

```

1 volatile bool dtcFaultPressed = false;
2 unsigned long lastDtcDebounce = 0;
3 bool canNoiseEnabled = true;
4
5 void onDtcFaultButton() {
6     dtcFaultPressed = true;
7 }
8
9 void setup() {
10     Serial.begin(SERIAL_BAUDRATE);
11     CAN.begin(CAN_SPEED);
12
13     pinMode(DTC_FAULT_PIN, INPUT_PULLUP);
14     attachInterrupt(digitalPinToInterrupt(DTC_FAULT_PIN),
15                     onDtcFaultButton, FALLING);
16
17     initVehicleData();
18 }
19
20 void loop() {

```

```

21  updatePhysicsModel();
22  handleDtcFaultButton();
23  generateBackgroundTraffic();
24
25  int packetSize = CAN.parsePacket();
26  if (packetSize > 0) {
27      processCANMessage();
28  }
29 }

```

Listado 3: Interrupción hardware para inyección de DTCs e inicialización

```

1  static const uint16_t DTC_POOL[] = {
2      0x0300, 0x0301, 0x0302, 0x0303,
3      0x0113, 0x0118, 0x0340, 0x0341,
4      0x0500, 0x0501, 0x0171, 0x0172
5  };
6  #define DTC_POOL_SIZE 12
7
8  void handleDtcFaultButton() {
9      if (!dtcFaultPressed) return;
10     dtcFaultPressed = false;
11
12     unsigned long now = millis();
13     if (now - lastDtcDebounce < DTC_DEBOUNCE_MS) return;
14     lastDtcDebounce = now;
15
16     uint16_t candidate = DTC_POOL[random(DTC_POOL_SIZE)];
17
18     // Deduplicacion: no insertar si ya esta presente
19     for (uint8_t i = 0; i < numStoredDTCs; i++) {
20         if (dtcList[i] == candidate) return;
21     }
22
23     if (numStoredDTCs < MAX_DTCS) {
24         dtcList[numStoredDTCs++] = candidate;
25         checkEngineLight = true;
26     }
27 }

```

Listado 4: Inyección de DTC por interrupción hardware con debounce

```

1  void processCANMessage() {
2      uint8_t buf[8];
3      int len = 0;
4      while (CAN.available() && len < 8) {
5          buf[len++] = CAN.read();
6      }
7
8      if (canNoiseEnabled) {
9          // Latencia variable: simula carga del bus
10         delay(random(CAN_NOISE_LATENCY_MIN_MS, CAN_NOISE_LATENCY_MAX_MS));
11
12         // Descarte aleatorio de paquetes (2%)
13         if (random(100) < CAN_NOISE_DROP_PCT) return;
14
15         // Respuesta negativa UDS esporadica (1%)
16         if (random(100) < CAN_NOISE_NRC_PCT) {
17             sendNegativeResponse(buf[1], NRC_CONDITIONS_NOT_CORRECT);

```



```

18     return;
19 }
20 }
21
22 // Decodificación del modo y PID
23 uint8_t mode = buf[1];
24 uint8_t pid = buf[2];
25 dispatchMode(mode, pid);
26 }
27
28 void generateBackgroundTraffic() {
29     if (!canNoiseEnabled) return;
30     if (random(100) >= CAN_NOISE_BG_TRAFFIC_PCT) return;
31
32     uint32_t id = CAN_BG_IDS[random(CAN_BG_ID_COUNT)];
33     CAN.beginPacket(id);
34     for (int i = 0; i < 8; i++) CAN.write(random(256));
35     CAN.endPacket();
36 }

```

Listado 5: Mecanismos de ruido CAN dentro del procesador de mensajes

```

1 void sendVIN() {
2     const char* vin = vehicleData.vin; // 17 bytes ASCII
3
4     // First Frame: PCI 0x10 0x14 (longitud total = 20 bytes)
5     uint8_t ff[8] = {
6         0x10, 0x14, // FF PCI: len=20
7         0x49, 0x02, 0x01, // modo respuesta, PID, count
8         (uint8_t)vin[0], (uint8_t)vin[1], (uint8_t)vin[2]
9     };
10    CAN.beginExtendedPacket(ECU_RESPONSE_ID);
11    for (int i = 0; i < 8; i++) CAN.write(ff[i]);
12    CAN.endPacket();
13
14    // Espera Flow Control del tester
15    waitFlowControl();
16
17    // Consecutive Frame 1: bytes VIN[3..9]
18    uint8_t cf1[8] = {0x21};
19    for (int i = 0; i < 7; i++) cf1[i+1] = (uint8_t)vin[3+i];
20    delay(ISOTP_CF_SEP_MS);
21    CAN.beginExtendedPacket(ECU_RESPONSE_ID);
22    for (int i = 0; i < 8; i++) CAN.write(cf1[i]);
23    CAN.endPacket();
24
25    // Consecutive Frame 2: bytes VIN[10..16] + padding
26    uint8_t cf2[8] = {0x22};
27    for (int i = 0; i < 7; i++) {
28        cf2[i+1] = (10+i < VIN_LENGTH) ? (uint8_t)vin[10+i] : ISOTP_PADDING_BYTE;
29    }
30    delay(ISOTP_CF_SEP_MS);
31    CAN.beginExtendedPacket(ECU_RESPONSE_ID);
32    for (int i = 0; i < 8; i++) CAN.write(cf2[i]);
33    CAN.endPacket();
34 }

```

Listado 6: Respuesta VIN con secuencia ISO-TP multi-frame

Anexo B — Código fuente de la herramienta Python

Este anexo recoge los fragmentos más representativos del código fuente de la herramienta de diagnóstico implementada en Python sobre Raspberry Pi.

B.1 — Interfaces abstractas del núcleo

```
1 from __future__ import annotations
2 from abc import ABC, abstractmethod
3 from types import TracebackType
4
5
6 class ITransport(ABC):
7
8     @abstractmethod
9     def connect(self) -> None: ...
10
11     @abstractmethod
12     def disconnect(self) -> None: ...
13
14     @abstractmethod
15     def send(self, payload: bytes) -> None: ...
16
17     @abstractmethod
18     def receive(self) -> bytes: ...
19
20     @abstractmethod
21     def __enter__(self) -> ITransport: ...
22
23     @abstractmethod
24     def __exit__(
25         self,
26         exc_type: type[BaseException] | None,
27         exc_val: BaseException | None,
28         exc_tb: TracebackType | None,
29     ) -> bool | None: ...
```

Listado 7: Interfaz ITransport — contrato de la capa de transporte

B.2 — Transporte ISO-TP sobre SocketCAN

```

1 import time
2 import can
3 import isotp
4 from config.can_config import CAN_RX_ID, CAN_TX_ID, ISOTP_CF_SEPARATION_MS, ISOTP_PADDING_BYTE
5 from core.exceptions import DiagnosticTimeoutError
6 from core.interfaces.i_transport import ITransport
7
8
9 class IsoTpTransport(ITransport):
10
11     def __init__(self, channel: str = "can0", timeout: float = 5.0) -> None:
12         self._channel = channel
13         self._timeout = timeout
14         self._address = isotp.Address(
15             addressing_mode=isotp.AddressingMode.Normal_11bits,
16             txid=CAN_TX_ID,
17             rxid=CAN_RX_ID,
18         )
19         self._params = {
20             "stmin": ISOTP_CF_SEPARATION_MS,
21             "blocksize": 0,
22             "tx_padding": ISOTP_PADDING_BYTE,
23         }
24         self._stack: isotp.CanStack | None = None
25         self._bus: can.BusABC | None = None
26
27     def connect(self) -> None:
28         self._bus = can.Bus(channel=self._channel, interface="socketcan")
29         # Vaciar tramas residuales del buffer del kernel
30         while self._bus.recv(timeout=0) is not None:
31             pass
32         self._stack = isotp.CanStack(self._bus, address=self._address,
33                                     params=self._params)
34         self._flush_rx()
35
36     def disconnect(self) -> None:
37         if self._stack is None:
38             return
39         self._bus.shutdown()
40         self._stack = None
41         self._bus = None
42
43     def send(self, payload: bytes) -> None:
44         self._flush_rx()
45         self._stack.send(payload)
46         while self._stack.transmitting():
47             self._stack.process()
48             time.sleep(0.001)
49
50     def receive(self) -> bytes:
51         deadline = time.monotonic() + self._timeout
52         while time.monotonic() < deadline:
53             self._stack.process()
54             if self._stack.available():
55                 return self._stack.recv()
56             time.sleep(0.001)

```

```

57         raise DiagnosticTimeoutError(f"No response within {self._timeout:.1f}s.")
58
59     def _flush_rx(self) -> None:
60         self._stack.process()
61         while self._stack.available():
62             self._stack.recv()
63             self._stack.process()

```

Listado 8: IsoTpTransport — implementacion concreta sobre python-can + can-isotp

B.3 — Logger SQLite con modo WAL

```

1 import sqlite3, threading
2 from datetime import datetime, timezone
3 from core.interfaces.i_data_logger import IDataLogger
4
5 _SCHEMA = """
6 PRAGMA journal_mode=WAL;
7
8 CREATE TABLE IF NOT EXISTS sessions (
9     id            INTEGER PRIMARY KEY AUTOINCREMENT,
10    label         TEXT      NOT NULL DEFAULT '',
11    started_at    TEXT      NOT NULL,
12    ended_at      TEXT
13 );
14
15 CREATE TABLE IF NOT EXISTS samples (
16     id            INTEGER PRIMARY KEY AUTOINCREMENT,
17     session_id    INTEGER NOT NULL REFERENCES sessions(id),
18     pid           INTEGER NOT NULL,
19     name          TEXT      NOT NULL,
20     value         REAL      NOT NULL,
21     unit          TEXT      NOT NULL,
22     monotonic_ts  REAL      NOT NULL,
23     wall_ts       TEXT      NOT NULL
24 );
25
26 CREATE TABLE IF NOT EXISTS commands (
27     id            INTEGER PRIMARY KEY AUTOINCREMENT,
28     session_id    INTEGER NOT NULL REFERENCES sessions(id),
29     command       TEXT      NOT NULL,
30     request_hex   TEXT      NOT NULL,
31     response_hex  TEXT      NOT NULL,
32     wall_ts       TEXT      NOT NULL
33 );
34 """
35
36 _BUFFER_SIZE = 50
37
38
39 class SqliteDataLogger(IDataLogger):
40     # WAL permite escrituras concurrentes desde hilo monitor y hilo BLE.
41
42     def __init__(self, db_path: str = "diagnostics.db") -> None:
43         self._lock = threading.Lock()

```

```

44     self._sample_buffer: list[tuple] = []
45     self._conn = sqlite3.connect(db_path, check_same_thread=False)
46     self._conn.executescript(_SCHEMA)
47     self._conn.commit()
48
49     def log_sample(self, session_id: int, sample) -> None:
50         row = (session_id, sample.pid, sample.name, sample.value,
51               sample.unit, sample.timestamp,
52               datetime.now(timezone.utc).isoformat())
53         with self._lock:
54             self._sample_buffer.append(row)
55             if len(self._sample_buffer) >= _BUFFER_SIZE:
56                 self._flush_buffer()
57
58     def log_command(self, session_id, command, request, response) -> None:
59         with self._lock:
60             self._conn.execute(
61                 "INSERT INTO commands"
62                 " (session_id, command, request_hex, response_hex, wall_ts)"
63                 " VALUES (?, ?, ?, ?, ?)",
64                 (session_id, command, request.hex(), response.hex(),
65                  datetime.now(timezone.utc).isoformat()),
66             )
67             self._conn.commit()
68
69     def _flush_buffer(self) -> None:
70         if not self._sample_buffer:
71             return
72         self._conn.executemany(
73             "INSERT INTO samples"
74             " (session_id, pid, name, value, unit, monotonic_ts, wall_ts)"
75             " VALUES (?, ?, ?, ?, ?, ?, ?)",
76             self._sample_buffer,
77         )
78         self._conn.commit()
79         self._sample_buffer.clear()

```

Listado 9: SqliteDataLogger — esquema, buffer de escritura y modo WAL

B.4 — Decorador LoggingTransport

```

1  from core.interfaces.i_transport import ITransport
2  from core.interfaces.i_data_logger import IDataLogger
3
4
5  class LoggingTransport(ITransport):
6      """Envuelve cualquier ITransport e intercepta send/receive para loggear
7      las tramas CAN en hexadecimal en SQLite sin modificar el transporte base."""
8
9      def __init__(self, inner: ITransport, logger: IDataLogger,
10                  session_id: int, command: str) -> None:
11         self._inner = inner
12         self._logger = logger
13         self._session_id = session_id
14         self._command = command

```

```
15         self._last_sent: bytes = b""
16
17     def connect(self) -> None:         self._inner.connect()
18     def disconnect(self) -> None:     self._inner.disconnect()
19     def __enter__(self):               self._inner.connect(); return self
20     def __exit__(self, *a):            self._inner.disconnect()
21
22     def send(self, payload: bytes) -> None:
23         self._last_sent = payload
24         self._inner.send(payload)
25
26     def receive(self) -> bytes:
27         response = self._inner.receive()
28         self._logger.log_command(
29             self._session_id, self._command,
30             self._last_sent, response,
31         )
32     return response
```

Listado 10: LoggingTransport — patron decorador sobre ITransport

Anexo C — Código fuente de la aplicación móvil

Este anexo recoge los fragmentos más representativos del adaptador BLE de la aplicación móvil React Native.

C.1 — Constantes NUS y conexión GATT

```
1 import { BleManager, Device, Subscription } from 'react-native-ble-plx';
2
3 const NUS_SERVICE = '6E400001-B5A3-F393-E0A9-E50E24DCCA9E';
4 const RX_CHAR      = '6E400002-B5A3-F393-E0A9-E50E24DCCA9E';
5 const TX_CHAR      = '6E400003-B5A3-F393-E0A9-E50E24DCCA9E';
6
7 const MTU           = 240;    // NUS safe max
8 const SCAN_TIMEOUT_MS = 20_000;
9 const REQUEST_TIMEOUT_MS = 15_000;
10 const PING_INTERVAL_MS = 7_000; // < server timeout (15s) con margen
11
12 export class BleAdapter implements IVehicleAdapter {
13   private static instance: BleAdapter;
14   private readonly manager = new BleManager();
15   private device: Device | null = null;
16   private txSub: Subscription | null = null;
17   private rxBuf = '';
18   private queue: Pending[] = [];
19
20   static getInstance(): BleAdapter {
21     if (!BleAdapter.instance) BleAdapter.instance = new BleAdapter();
22     return BleAdapter.instance;
23   }
24 }
```

Listado 11: UUIDs del servicio NUS y configuración de la conexión BLE

```
1 async connect(deviceId?: string): Promise<void> {
2   if (!deviceId) throw new Error('No device selected');
3
4   // 1. Conexión GATT física
5   const raw = await this.manager.connectToDevice(deviceId,
6     { autoConnect: false, timeout: 10000 });
7 }
```

```

8  // 2. Negociacion MTU
9  const mtuResult = await raw.requestMTU(MTU);
10
11 // 3. Descubrimiento de servicios y características
12 await raw.discoverAllServicesAndCharacteristics();
13
14 // 4. Registro de callback de desconexion inesperada
15 raw.onDisconnected(() => this.handleUnexpectedDisconnect());
16 this.device = raw;
17
18 // 5. Suscripcion a notificaciones TX (RX desde perspectiva del servidor)
19 await this.subscribeToTx();
20
21 // 6. Autenticacion por token
22 await this.request({ cmd: 'auth', token: '1234' });
23
24 this.startPingInterval();
25 this.startActivityMonitor();
26 }

```

Listado 12: Flujo de conexion: GATT, MTU, servicios, suscripcion y autenticacion

C.2 — Buffer NDJSON y despachador de mensajes

```

1 private feedBuffer(chunk: string): void {
2   this.lastActivityTime = Date.now();
3   this.rxBuf += chunk;
4   // Separar por lineas completas; la ultima puede estar incompleta
5   const lines = this.rxBuf.split('\n');
6   this.rxBuf = lines.pop() ?? '';
7
8   for (const line of lines) {
9     const t = line.trim();
10    if (!t) continue;
11    try {
12      const parsed = JSON.parse(t) as Record<string, unknown>;
13      this.dispatch(parsed);
14    } catch {
15      // JSON malformado -- descartar fragmento
16    }
17  }
18 }
19
20 private dispatch(msg: Record<string, unknown>): void {
21   // Muestras de monitorizacion continua: enviar a todos los callbacks
22   if (msg.type === 'samples') {
23     const batch = msg.samples as MonitorSample[];
24     batch.forEach((s) => this.sampleCbs.forEach((cb) => cb(s)));
25     return;
26   }
27
28   // Heartbeat del servidor: responder con ack
29   if (msg.type === 'heartbeat') {
30     this.sendHeartbeatAck();
31     return;

```



```

32 }
33
34 // Respuesta a comando: resolver o rechazar la promesa en cola
35 const pending = this.queue.shift();
36 if (!pending) return;
37 clearTimeout(pending.timer);
38
39 if (msg.status === 'ok') {
40     pending.resolve(msg.data);
41 } else {
42     pending.reject(new Error((msg.message as string) ?? 'Server error'));
43 }
44 }

```

Listado 13: Procesamiento del buffer NDJSON recibido por notificaciones BLE

C.3 — Escritura BLE con chunking y timeout

```

1 private request<T = unknown>(cmd: object): Promise<T> {
2     return new Promise<T>((resolve, reject) => {
3         const json = JSON.stringify(cmd) + '\n';
4
5         // Timeout por si la Raspberry Pi no responde
6         const timer = setTimeout(() => {
7             const idx = this.queue.findIndex((p) => p.timer === timer);
8             if (idx !== -1) this.queue.splice(idx, 1);
9             reject(new Error('Timeout: ${JSON.stringify(cmd)}'));
10        }, REQUEST_TIMEOUT_MS);
11
12        this.queue.push({ resolve: resolve as (d: unknown) => void, reject, timer });
13        this.writeRx(json).catch((e) => {
14            const idx = this.queue.findIndex((p) => p.timer === timer);
15            if (idx !== -1) { clearTimeout(timer); this.queue.splice(idx, 1); }
16            reject(e);
17        });
18    });
19 }
20
21 private async writeRx(data: string, withResponse = true): Promise<void> {
22     if (!this.device) throw new Error('Not connected');
23     const bytes = new TextEncoder().encode(data);
24
25     // Fragmentar en chunks de MTU bytes para el caso de payloads grandes
26     for (let offset = 0; offset < bytes.length; offset += MTU) {
27         const slice = bytes.subarray(offset, offset + MTU);
28         let bin = '';
29         for (let i = 0; i < slice.length; i++) bin += String.fromCharCode(slice[i]);
30         const b64 = btoa(bin);
31
32         if (withResponse) {
33             await this.device.writeCharacteristicWithResponseForService(
34                 NUS_SERVICE, RX_CHAR, b64);
35         } else {
36             await this.device.writeCharacteristicWithoutResponseForService(
37                 NUS_SERVICE, RX_CHAR, b64);

```

```

38     }
39   }
40 }

```

Listado 14: Sistema de peticion-respuesta con timeout y chunking por MTU

C.4 — Keepalive y monitor de inactividad

```

1 private startPingInterval(): void {
2   this.pingInterval = setInterval(() => {
3     if (!this.device) return;
4     // Write sin respuesta: el pong se descarta silenciosamente en dispatch()
5     this.writeRx(JSON.stringify({ cmd: 'ping' }) + '\n', false).catch(() => {});
6   }, PING_INTERVAL_MS);
7 }
8
9 private startActivityMonitor(): void {
10  this.activityMonitorInterval = setInterval(() => {
11    if (!this.device) return;
12    const elapsed = Date.now() - this.lastActivityTime;
13    if (elapsed > CLIENT_TIMEOUT_MS) {
14      this.disconnect().catch(() => {});
15    }
16  }, INACTIVITY_CHECK_MS);
17 }
18
19 private handleUnexpectedDisconnect(): void {
20   if (this.pingInterval) clearInterval(this.pingInterval);
21   if (this.activityMonitorInterval) clearInterval(this.activityMonitorInterval);
22   this.txSub?.remove();
23   this.txSub = null;
24   this.rxBuf = '';
25   this.device = null;
26   this.drainQueue(new Error('Device disconnected unexpectedly'));
27 }

```

Listado 15: Ping periodico y desconexion automatica por inactividad

Anexo D — Esquema de la base de datos SQLite

La base de datos SQLite empleada para la persistencia de datos de diagnóstico se compone de tres tablas relacionadas. El esquema completo, incluyendo los índices y la configuración WAL, se muestra a continuación.

D.1 — Esquema SQL

```
1 PRAGMA journal_mode=WAL;
2
3 -- Sesiones de diagnostico
4 CREATE TABLE IF NOT EXISTS sessions (
5     id            INTEGER PRIMARY KEY AUTOINCREMENT,
6     label         TEXT      NOT NULL DEFAULT '',
7     started_at    TEXT      NOT NULL,    -- ISO 8601 UTC
8     ended_at      TEXT      -- NULL si la sesion sigue activa
9 );
10
11 -- Muestras de telemetria OBD-II
12 CREATE TABLE IF NOT EXISTS samples (
13     id            INTEGER PRIMARY KEY AUTOINCREMENT,
14     session_id    INTEGER NOT NULL REFERENCES sessions(id),
15     pid           INTEGER NOT NULL,      -- PID OBD-II en decimal
16     name          TEXT      NOT NULL,    -- nombre legible del parametro
17     value         REAL      NOT NULL,    -- valor decodificado con unidades
18     unit          TEXT      NOT NULL,    -- unidad de medida (rpm, km/h, ...)
19     monotonic_ts  REAL      NOT NULL,    -- time.monotonic() para ordenacion
20     wall_ts       TEXT      NOT NULL     -- ISO 8601 UTC para presentacion
21 );
22
23 -- Comandos ejecutados y tramas CAN asociadas
24 CREATE TABLE IF NOT EXISTS commands (
25     id            INTEGER PRIMARY KEY AUTOINCREMENT,
26     session_id    INTEGER NOT NULL REFERENCES sessions(id),
27     command       TEXT      NOT NULL,    -- nombre del comando (query, dtcs, vin...)
28     request_hex   TEXT      NOT NULL,    -- trama ISO-TP de peticion en hex
29     response_hex  TEXT      NOT NULL,    -- trama ISO-TP de respuesta en hex
30     wall_ts       TEXT      NOT NULL     -- ISO 8601 UTC
31 );
32
33 -- Indices para consultas por sesion
```

```

34 CREATE INDEX IF NOT EXISTS idx_samples_session
35     ON samples(session_id, pid);
36
37 CREATE INDEX IF NOT EXISTS idx_commands_session
38     ON commands(session_id);

```

Listado 16: Esquema completo de la base de datos diagnostics.db

D.2 — Descripción de las tablas

La tabla **sessions** registra cada sesión de diagnóstico con sus timestamps de inicio y fin. El campo `ended_at` es NULL mientras la sesión está activa y se actualiza al cerrarla mediante `end_session()`.

La tabla **samples** almacena cada muestra individual de telemetría. El campo `pid` permite filtrar las muestras de un PID concreto dentro de una sesión, y el campo `request_hex` / `response_hex` en la tabla **commands** almacena las tramas CAN en formato hexadecimal compacto (sin espacios), lo que permite reconstruir la secuencia ISO-TP completa para cada comando ejecutado.

La tabla **commands** registra todos los comandos ejecutados desde la aplicación móvil, incluyendo los de gestión de DTCs y obtención de VIN. El campo `response_hex` de un comando de modo 0x09 (VIN) contiene la concatenación de la First Frame y las Consecutive Frames recibidas.

D.3 — Ejemplo de contenido

La tabla 1 muestra un fragmento representativo del contenido de la tabla **samples** tras una sesión de monitorización de cuatro PIDs.

Cuadro 1: Fragmento de la tabla **samples** — sesión de ejemplo.

id	pid	name	value	unit	wall_ts
1	12	rpm	1724.0	rpm	2025-04-10T10:23:01Z
2	13	vehicle_speed	87.0	km/h	2025-04-10T10:23:01Z
3	5	coolant_temp	91.0	°C	2025-04-10T10:23:02Z
4	17	throttle_pos	23.5	%	2025-04-10T10:23:02Z
5	12	rpm	1756.0	rpm	2025-04-10T10:23:02Z

D.4 — Consideraciones de concurrencia

La activación del modo WAL (*Write-Ahead Logging*) mediante `PRAGMA journal_mode=WAL` es esencial para el correcto funcionamiento concurrente del sistema. Sin WAL, las escrituras del hilo de monitorización bloquearían las lecturas del hilo BLE durante el volcado del buffer, introduciendo latencias variables en las respuestas a la aplicación móvil. Con WAL, los lectores no se bloquean por los escritores en ningún caso, al operar sobre versiones consistentes del fichero de base de datos.

El buffer de escritura de 50 muestras implementado en `SqliteDataLogger` reduce la frecuencia de escrituras a disco a una transacción cada 50 muestras (aproximadamente cada 25 segundos a 500 ms de periodo de muestreo con cuatro PIDs activos), minimizando el tiempo de bloqueo del *write lock* WAL y el desgaste de la tarjeta SD de la Raspberry Pi.

Bibliografía

- [1] Robert Bosch GmbH, *CAN Specification Version 2.0*, Robert Bosch GmbH, Stuttgart, 1991.
- [2] International Organization for Standardization, *ISO 11898-1:2015 – Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling*, ISO, Ginebra, 2015.
- [3] International Organization for Standardization, *ISO 15765-2:2016 – Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) – Part 2: Transport protocol and network layer services*, ISO, Ginebra, 2016.
- [4] Society of Automotive Engineers, *SAE J1979:2012 – E/E Diagnostic Test Modes*, SAE International, Warrendale, 2012.
- [5] International Organization for Standardization, *ISO 14229-1:2020 – Road vehicles – Unified diagnostic services (UDS) – Part 1: Application layer*, ISO, Ginebra, 2020.
- [6] S. Ménager *et al.*, *python-can-isotp Documentation*, 2023. Disponible en: <https://can-isotp.readthedocs.io/>
- [7] python-can contributors, *python-can Documentation*, 2023. Disponible en: <https://python-can.readthedocs.io/>